

What is a graph data structure?

There are two types of data structures

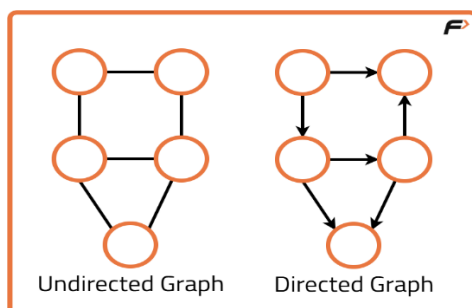
- Linear
- Non - linear

We are aware of linear data structures such as arrays, stacks, queues, and linked lists. They are called linear because data elements are arranged in a linear or sequential manner.

The only non-linear data structure that we've seen so far is Tree. In fact, a tree is a special type of graph with some restrictions. Graphs are data structures that have a wide-ranging application in real life. These include analysis of electrical circuits, finding the shortest routes between two places, building navigation systems like Google Maps, even social media using graphs to store data about each user, etc. To understand and use the graph data structure, let's get familiar with the definitions and terms associated with graphs.

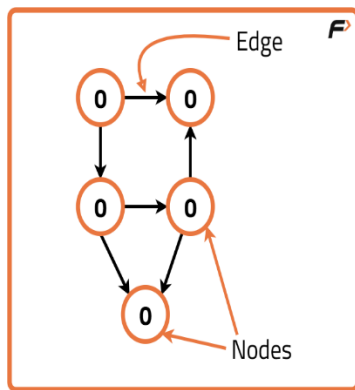
Definitions and Terminology

A graph is a non-linear data structure consisting of nodes that have data and are connected to other nodes through edges.



Nodes are circles represented by numbers. Nodes are also referred to as vertices. They store the data. The numbering of the nodes can be done in any order, no specific order needs to be followed.

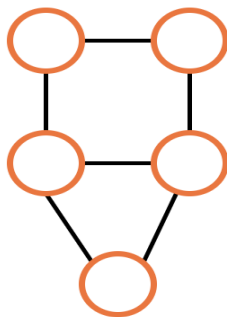
In the following example, the number of nodes or vertices = 5.



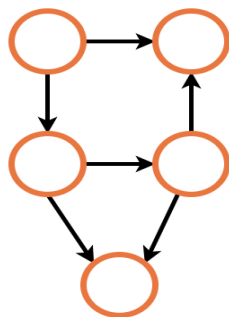
Two nodes are connected by a horizontal line called **Edge**. Edge can be directed or undirected. Basically, pairs of vertices are called edges.

In the above example, the edge can go from 1 to 4 or from 4 to 1, i.e. a bidirectional edge can be in both directions, hence called an **undirected edge**. Thus, the pairs (1,4) and (4,1) represent the same edge.

Types of Graphs



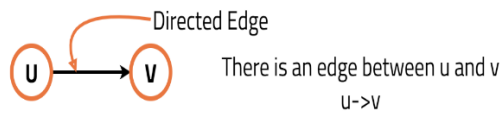
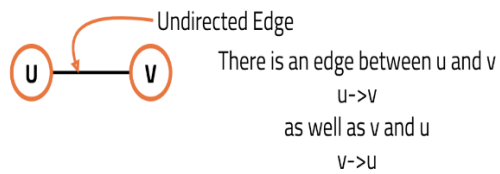
Undirected Graph



Directed Graph

- **An undirected graph** is a graph where edges are bidirectional, with no direction associated with them, i.e, there will be an undirected edge. In an undirected graph, the pair of vertices representing any edge is unordered. Thus, the pairs (u, v) and (v, u) represent the same edge.
- **A directed graph** is a graph where all the edges are directed from one vertex to another, i.e, there will be a directed edge. It contains an ordered pair of vertices. It implies each edge is represented by a directed pair $\langle u, v \rangle$. Therefore, $\langle u, v \rangle$ and $\langle v, u \rangle$ represent two different edges.

There can be multi-directed edges, hence bidirectional edges, as shown in the example below.



Structure of Graph

Does every graph have a cycle?

The answer is No! Let us consider the following examples to understand this.

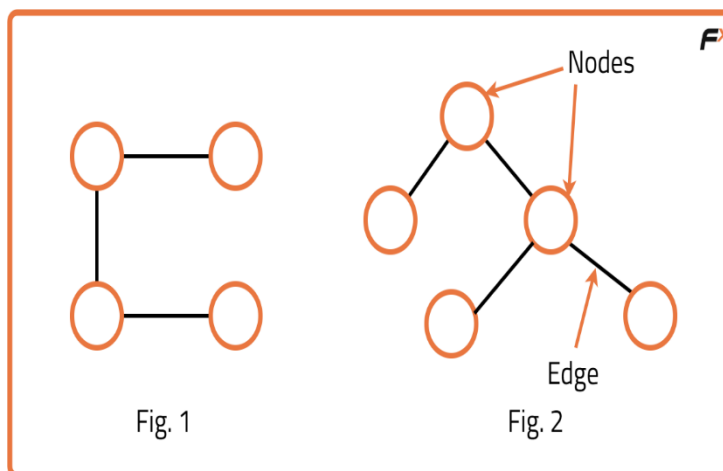
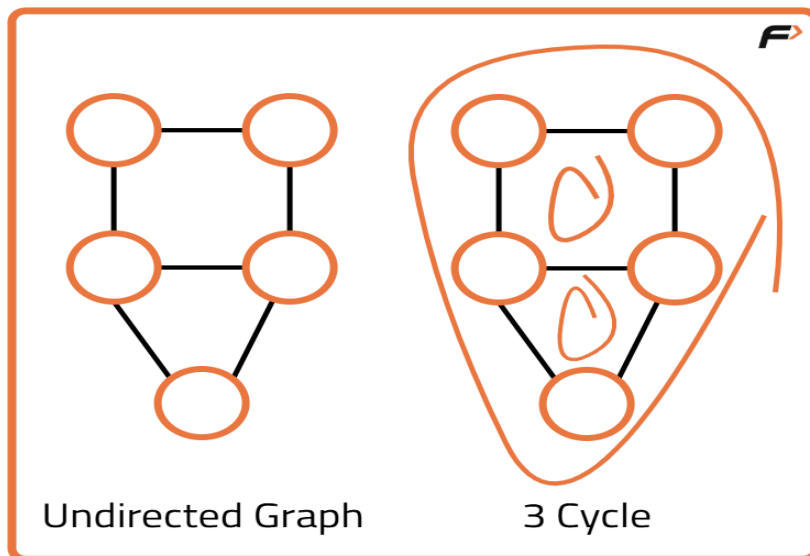


Fig. 1 does not form a cycle but still, it is a graph.

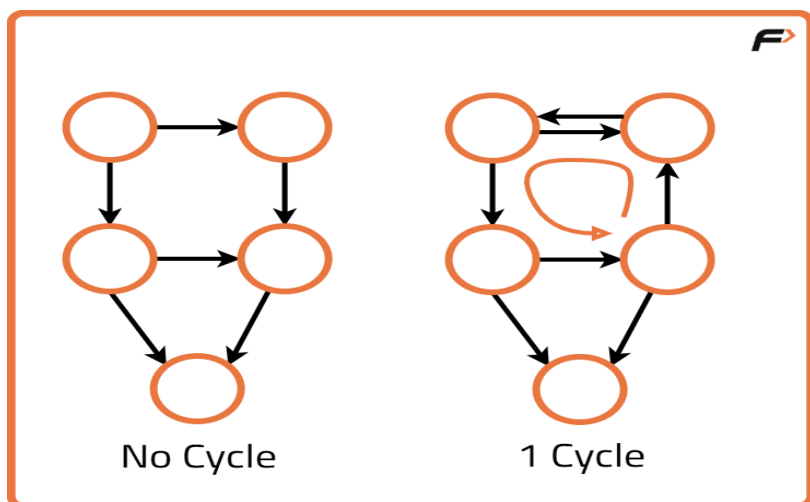
Fig. 2 is an example of a binary tree. It can also be called a graph because it follows all the rules. We've nodes and edges, and this is the minimal condition to be called a graph.

So, a graph does not necessarily mean to be an enclosed structure, it can be an open structure as well. A graph is said to have a cycle if it starts from a node and ends at the same node. There can be multiple cycles in a graph.



If there is at least one cycle present in the graph then it is called an **Undirected Cyclic Graph**.

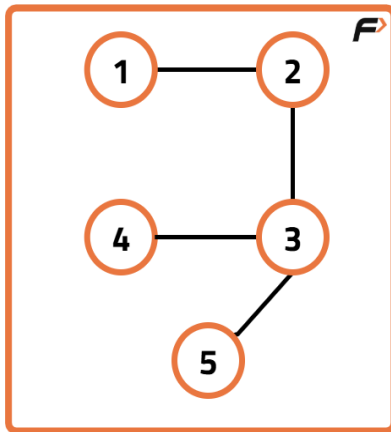
In the following examples of directed graphs, the first directed graph is not cyclic as we can't start from a node and end at the same node. Hence it is called **Directed Acyclic Graph**, commonly called **DAG**.



If we just add an edge to the directed graph, then at least one cycle is present in the graph, hence it becomes **Directed Cyclic Graph**.

Path in a Graph: The path contains a lot of nodes and each of them is reachable.

Consider the given graph,



1 2 3 5 is a path.

1 2 3 2 1 is not a path, because a node can't appear twice in a path.

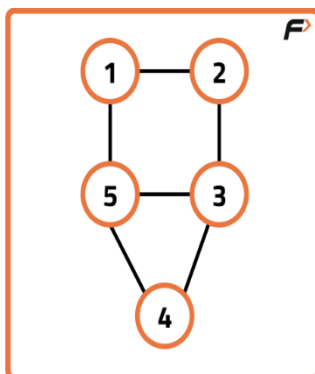
1 3 5 is not a path, as adjacent nodes must have an edge and there is no edge between 1 and 3.

Degree of Graph

It is the number of edges that go inside or outside that node.

For **undirected graphs**, the degree is the number of edges attached to a node.

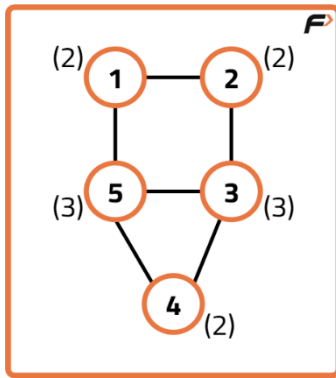
Example,



$$D(3) = 3$$

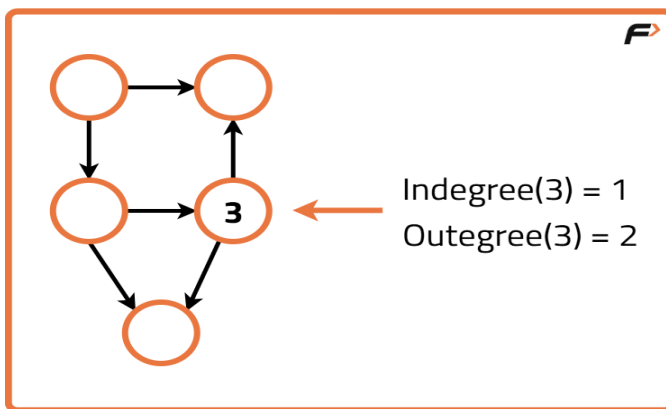
$$D(4) = 2$$

Property: It states that the total degree of a graph is equal to twice the number of edges. This is because every edge is associated/ connected to two nodes.



Total Degree of a graph = $2 \times E$

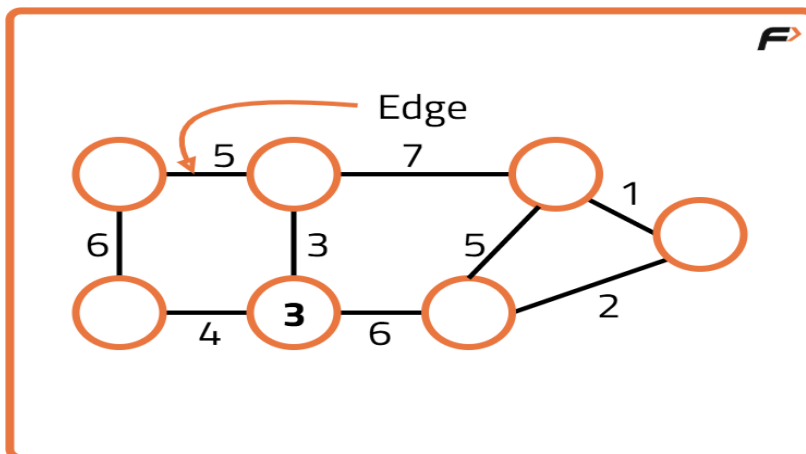
Example, $(2+2+3+2+3) = 2 \times 6 \Rightarrow 12 = 12$



For **directed graphs**, we've Indegree and Outdegree. **The indegree** of a node is the number of incoming edges. **The outdegree** of a node is the number of outgoing edges.

Edge Weight

A graph may have weights assigned on its edges. It is often referred to as the cost of the edge.



If weights are not assigned then we assume the unit weight, i.e., 1. In applications, weight may be a measure of the cost of a route. For example, if vertices A and B represent towns in a road network, then weight on edge AB may represent the cost of moving from A to B, or vice versa.

Graph Representation in Java

Input Format

In the question, they will mention whether it is a directed or undirected graph. The first line contains two space-separated integers n and m denoting the number of nodes and the number of edges respectively. Next m lines contain two integers u and v representing an edge between u and v . In the case of an undirected graph if there is an edge between u and v , it means there is an edge between v and u as well. Now the question arises if there is any bound on the number of edges, i.e., the value of m ? The answer is NO. If we add more edges, then the value of m will increase.

The two most commonly used representations for graphs are

- Adjacency Matrix
- Adjacency Lists

Adjacency Matrix

An adjacency matrix of a graph is a two-dimensional array of size $n \times n$, where n is the number of nodes in the graph, with the property that $a[i][j] = 1$ if the edge (v_i, v_j) is in the set of edges, and $a[i][j] = 0$ if there is no such edge.

Consider the example of the following undirected graph,

Input:

5 6

1 2

1 3

2 4

3 4

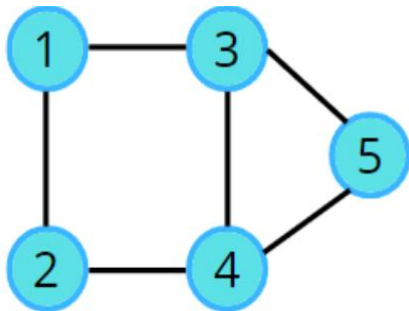
3 5

Explanation:

Number of nodes, $n = 5$

Number of edges, $m = 6$

Next 'm' lines represent the edges.



We need to store these edges so that future algorithms can be performed. Are the nodes zero-based or one-based? In this case, the nodes follow one-based indexing as the last node is 5 and the total number of nodes is also 5. Now, define an adjacency matrix of size $(n+1) \times (n+1)$, i.e., $\text{adj}[n+1][n+1]$. If there is an edge between 1 and 2, mark 1 at (1,2) and (2,1) as there is an edge between 2 and 1 as well (in the case of an undirected graph). Similarly, follow for other edges.

All the edges are marked in the adjacency matrix, remaining spaces in the matrix are marked as zero or left as it is.

	0	1	2	3	4	5
0						
1			1	1		
2		1			1	
3		1			1	1
4			1	1		1
5				1	1	

This matrix will tell if there is an edge between two particular nodes. For example, there is an edge between 5 and 3 as 1 is at (5,3) but there is no edge between 5 and 1 as the space is empty (or can be filled with 0) at position (5,1) in the adjacency matrix.

The space needed to represent a graph using its adjacency matrix is n^2 locations.

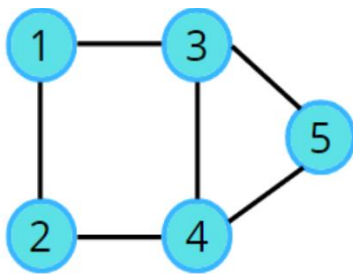
Space complexity = $(n*n)$, It is a costly method as n^2 locations are consumed.

Adjacency Lists

In the previous storing method, we saw it was taking n^2 space to store the graph, this is where the adjacency list comes into the picture, it takes a very less amount of space.

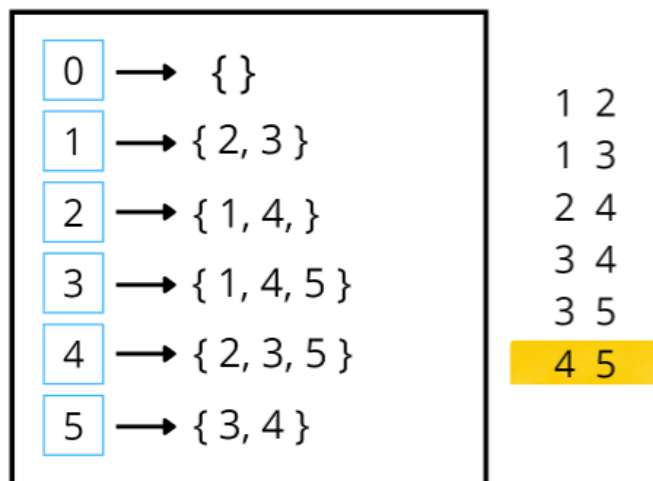
This is a node-based representation. In this representation, we use a 2D ArrayList.

Consider the example of the following **undirected graph**,



To create an adjacency list, we will create a 2-D ArrayList of size $n+1$ where n is the number of nodes.

```
ArrayList<ArrayList<Integer>> list = new ArrayList<>();  
for (int i=0;i<=nodes;i++){  
    list.add(new ArrayList<>());  
}
```



Hence, we stored all the neighbors in the particular indexes. In this representation, for an undirected graph, each edge data appears twice. For example, nodes 1 and 2 are adjacent hence node 2 appears in the list of node 1, and node 1 appears in the list of node 2. So, the

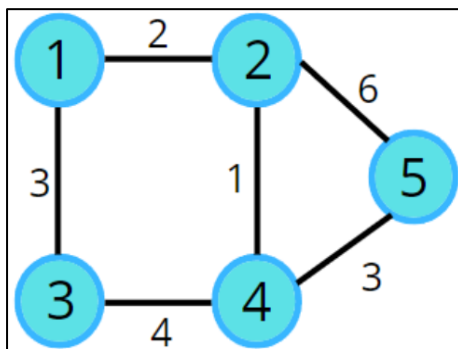
space needed to represent an undirected graph using its adjacency list is **2 x Edges**, where E denotes the number of edges. The Space Complexity is **$O(2 \cdot E)$** .

This representation is much better than the adjacency matrix, as matrix representation consumes n^2 locations, and most of them are unused.

For directed graphs, if there is an edge between u and v it means the edge only goes from u to v, i.e., v is the neighbor of u, but vice versa is not true. The space needed to represent a directed graph using its adjacency list is E locations, where E denotes the number of edges, as here each edge data appears only once. The space complexity is $O(E)$.

Weighted Graph Representation

As of now, we were considering graphs with unit weight edges (i.e., if there is an edge between two nodes then the weight on the edge is unit weight), now what if there are weights on its edges as shown in the following example?



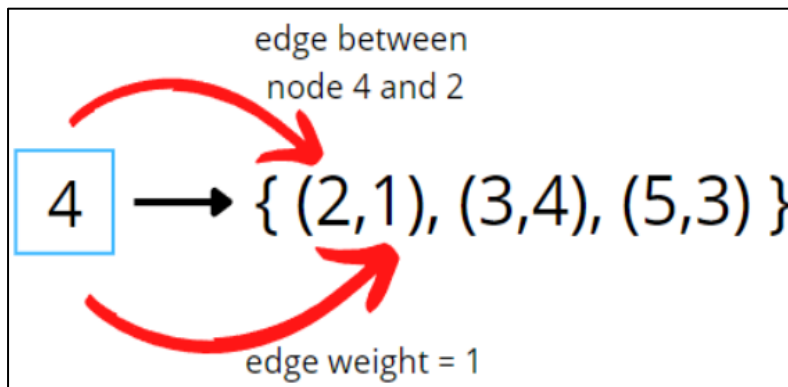
For the **adjacency matrix**, it is much simpler.

Undirected Graph	Directed Graph
<pre>int u, v, wt; cin >> u >> v >> wt; adj[u][v] = wt; adj[v][u] = wt;</pre>	<pre>int u, v, wt; cin >> u >> v >> wt; adj[u][v] = wt;</pre>

But how are we going to implement it in the **adjacency list**?

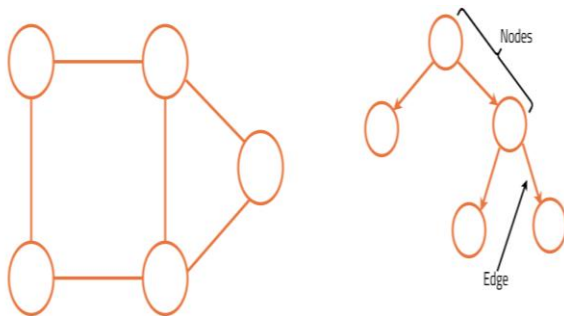
Earlier in the adjacency list, we were storing a list of integers in each index, but for weighted graphs, we will store pairs (node, edge weight) in it.

```
ArrayList<Pair<Integer, Integer>>[] adj = new ArrayList[n+1];
```

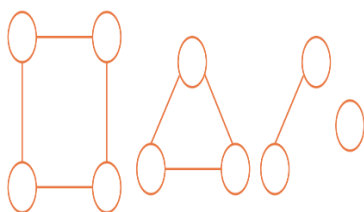


Connected Components in Graphs

So, far we've seen different types of graphs. Graphs can be connected or can be like a binary tree (as we know all trees are graphs with some restrictions) as shown in the following figure.



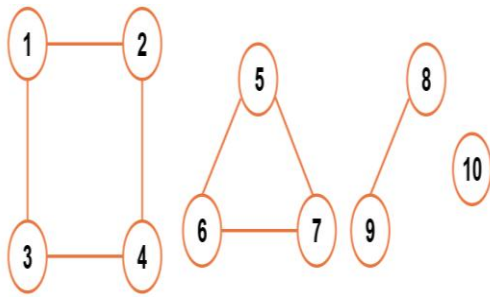
But what would you call the following figure?



The most common answer would be these are 4 different graphs as they are not connected.

But is it possible to call them a single graph? To answer this, let us consider the question given:

Given an undirected graph with 10 nodes and 8 edges. The edges are (1,2), (1,3), (2,4), (4,3), (5,6), (5,7), (6,7), (8,9). The graph that can be formed with the given information is as follows:



Apparently, it's a graph, which is in 4 pieces, the last one being a single node. In this case, we can say, the graph has been broken down into 4 different **connected components**. So next time if you see two different parts of a graph and they are not connected, then do not say that it cannot be a single graph. In the above example, they can be 4 different graphs but according to the given question and the input, we can call them parts of a single graph.

Graph Traversal

In the upcoming topics, we'll be learning about a lot of algorithms. Now, assume a traversal algorithm. Any traversal algorithm will always use a **visited array**.

For the same example, we will create an array of size 11 (number of nodes +1) starting with the zeroth index. Initialize this visited array to zero, indicating that all the nodes are unvisited. Then follow the following algorithm. If a node is not visited, then call the traversal algorithm.

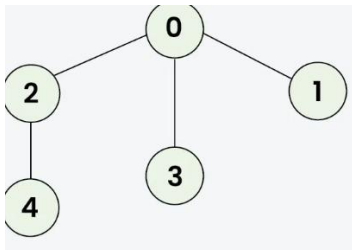
```
for(int i=1;i<=number of nodes;i++){
    if(!visited[i]) traversal(i);
}
```

Why can't we just call traversal(1)?

We cannot just call traversal(node) because a graph can have multiple components and traversal algorithms are designed in such a way that they will traverse the entire connected portion of the graph. For example, traversal(1) will traverse only the connected nodes, i.e., nodes 2, 3, and 4, but not the connected components.

Given a connected undirected graph containing V vertices, represented by a 2-d adjacency list adj[[]], where each adj[i] represents the list of vertices connected to vertex i. Perform a Breadth First Search (BFS) traversal starting from vertex 0, visiting vertices from left to right according to the given adjacency list, and return a list containing the BFS traversal of the graph.

Input: adj[][] = [[2, 3, 1], [0], [0, 4], [0], [2]]



Output: [0, 2, 3, 1, 4]

Approach:

- Keep a queue data structure & unvisited array of size equal to adjList size.
- Start from node 0. Push it & mark it visited.
- Take one node at a time from the queue & mark it visited. Add it to result.
- Push all its unvisited neighbours.
- Continue until no nodes remain in the queue.

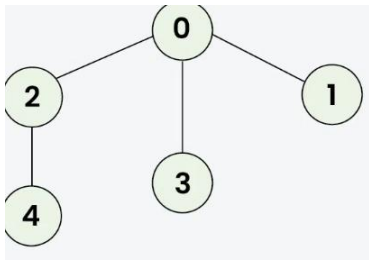
```
public ArrayList<Integer> bfs(ArrayList<ArrayList<Integer>> adjList) {  
    // code here  
    ArrayList<Integer> traversal = new ArrayList<>();  
    int[] visited = new int[adjList.size()];  
  
    int startingNode = 0;  
    Queue<Integer> q = new LinkedList<>();  
  
    q.add(startingNode);  
    visited[startingNode]=1;  
    while(!q.isEmpty()){  
        int curr = q.poll();  
        traversal.add(curr);  
        for (int e: adjList.get(curr)){  
            if (visited[e] == 0){  
                q.add(e);  
                visited[e] = 1;  
            }  
        }  
    }  
    return traversal;  
}
```

Time Complexity: $O(N) + O(2E)$, Where N = Nodes, $2E$ is for total degrees as we traverse all adjacent nodes.

Space Complexity: $O(3N) \sim O(N)$, Space for queue data structure visited array and an traversal array-list.

Given a connected undirected graph containing V vertices represented by a 2-d adjacency list adj[][], where each adj[i] represents the list of vertices connected to vertex i. Perform a Depth First Search (DFS) traversal starting from vertex 0, visiting vertices from left to right as per the given adjacency list, and return a list containing the DFS traversal of the graph.

Input: `adj[][] = [[2, 3, 1], [0], [0, 4], [0], [2]]`



Output: `[0, 2, 4, 3, 1]`

Implementation:

- Create a traversal list to store the DFS order.
- Create a visited array to avoid revisiting nodes.
- Initialize a stack and push the starting node (0).
- Process until the stack is empty:
 - Pop the top node.
 - If already visited → skip it.
 - Mark it visited and add it to traversal.
 - Push all its neighbours in reverse order so they are visited in the original left-to-right order.
- Return the traversal list after completing the DFS.

```

class Solution {
    public ArrayList<Integer> dfs(ArrayList<ArrayList<Integer>> adjList) {
        // code here

        ArrayList<Integer> traversal = new ArrayList<>();
        int[] visited = new int[adjList.size()];

        int startingNode = 0;
        Stack<Integer> st = new Stack<>();
        st.push(startingNode);

        while (!st.isEmpty()){
            int curr = st.pop();
            if(visited[curr]==1) continue;

            visited[curr]=1;
            traversal.add(curr);

            for(int i=adjList.get(curr).size()-1;i>=0;i--){
                int node = adjList.get(curr).get(i);
                if(visited[node]==0){
                    st.push(node);
                }
            }
        }
        return traversal;
    }
}

```

Time Complexity: $O(N) + O(2E)$, Where N = Nodes, $2E$ is for total degrees as we traverse all adjacent nodes.

Space Complexity: $O(3N) \sim O(N)$, Space for stack data structure visited array and an traversal array-list.

0/1 BFS:

- Variation of simple BFS.
- Used to shortest path from source to all other nodes in a weighted graph & weight between edges be either 0 or 1 without using priority queue dijkstra algorithm.
- In normal BFS, at any point of time there will be nodes of only two levels.
- In order to implement 0/1 bfs will use double ended queue that is deque. In which we add the (node,weight) at the front if weight==0, Otherwise will add at the last. In this way we can maintain the sorted order. That's why we don't need the priority queue.
- In this we will also get the element from front in each step.

Problems On BFS And DFS

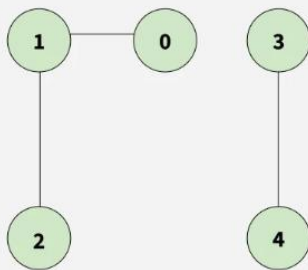
Given an undirected graph with V vertices numbered from 0 to $V-1$ and E edges, represented as a 2D array `edges[][]`, where each entry `edges[i] = [u, v]` denotes an edge between vertices u and v .

Your task is to return a list of all connected components. Each connected component should be represented as a list of its vertices, with all components returned in a collection where each component is listed separately.

Input: $V = 5$, `edges[][] = [[0, 1], [2, 1], [3, 4]]`

Output: `[[0, 1, 2], [3, 4]]`

Explanation:



★ Intuition (Short)

A graph may have several separate groups of nodes (connected components).

To find each component:

- Start BFS from an unvisited node → this will collect all nodes connected to it.
- Mark all of them visited.
- Repeat the process for the next unvisited node.
- Each BFS gives one complete connected component.

★ Implementation Steps:

1. Build adjacency list

Convert the edge list into adjacency lists so you can quickly access neighbors.

2. Maintain a visited array

Track which nodes have already been included in some component.

3. Loop through all nodes

For each node:

- If it is unvisited → start a BFS/DFS from it.
- This BFS/DFS will collect all nodes belonging to that connected component.

5. Add the BFS/DFS result to the final answer

Each BFS result list is one component.

6. Return the list of all components

```
public ArrayList<ArrayList<Integer>> getComponents(int n, int[][] edges) {
    ArrayList<ArrayList<Integer>> adjList = new ArrayList<>();
    for (int i=0;i<n;i++) adjList.add(new ArrayList<>());
    for (int i=0;i<edges.length;i++){
        int n1 = edges[i][0];
        int n2 = edges[i][1];
        adjList.get(n1).add(n2);
        adjList.get(n2).add(n1);
    }
    int[] visited = new int[n];
    ArrayList<ArrayList<Integer>> res = new ArrayList<>();
    for(int i=0;i<n;i++){
        if(visited[i]==0){
            ArrayList<Integer> l = traversal(i,visited,adjList);
            res.add(l);
        }
    }
    return res;
}
```

```
ArrayList<Integer> traversal(int node,int[] visited, ArrayList<ArrayList<Integer>> adjList){
    Queue<Integer> q = new LinkedList<>();
    ArrayList<Integer> l = new ArrayList<>();
    q.add(node);
    visited[node]=1;
    while(!q.isEmpty()){
        int curr = q.poll();
        l.add(curr);
        for (int e: adjList.get(curr)){
            if (visited[e] == 0){
                q.add(e);
                visited[e] = 1;
            }
        }
    }
    return l;
}
```

Time: $O(V+E) + O(V) + O(V+2E)$ ($V+E$) for creating adjList, $O(V)$ for visited array, & $O(V+2E)$ for doing traversal.

Space: $O(V + E) + 3O(V)$ ($V+E$) for adjList, $\sim O(V)$ for visited array, queue & result list.

Where V=nodes & E=edges.

Given a grid of size $n \times m$ (n is the number of rows and m is the number of columns in the grid) consisting of 'W's (Water) and 'L's (Land). Find the number of islands.

Note: An island is either surrounded by water or the boundary of a grid and is formed by connecting adjacent lands horizontally or vertically or diagonally i.e., in all 8 directions.

Input: `grid[][] = [['L', 'L', 'W', 'W', 'W'],`
 `['W', 'L', 'W', 'W', 'L'],`
 `['L', 'W', 'W', 'L', 'L'],`
 `['W', 'W', 'W', 'W', 'W'],`
 `['L', 'W', 'L', 'L', 'W']]`

Output: 4

Explanation:

The image below shows all the 4 islands in the grid.

L	L	W	W	W
W	L	W	W	L
L	W	W	L	L
W	W	W	W	W
L	W	L	L	W

Optimal approach:

Intuition

Think of the matrix as a map:

- 'L' = land
- 'W' = water

An island is a group of land cells ('L') that are connected to each other.

Connection is allowed in all 8 directions (up, down, left, right, and diagonals).

Core idea:

- Every time you find an unvisited land cell, you have discovered a new island.
- From that cell, use BFS to visit all connected land cells.
- Mark them visited so they are not counted again.
- Repeat this for the entire grid.

Each BFS covers one complete island.

Implementation Steps

1 Create a visited matrix

- Same size as the input matrix.
 - Keeps track of land cells already explored.
-

2 Traverse every cell of the matrix

- Use two nested loops to scan the grid.
 - When you find a cell that is:
 - 'L' (land)
 - not visited
→ this marks the start of a new island.
-

3 Increase-island count

- Each new unvisited land cell means one new island.
-

4 Perform BFS from that cell

- Push the starting cell into a queue.
 - Mark it as visited.
 - While the queue is not empty:
 - Remove the current cell.
 - Explore all 8 neighboring cells.
 - If a neighbor is inside the grid, is land, and unvisited:
 - Mark it visited.
 - Add it to the queue.
-

5 Continue scanning the grid

- BFS marks the whole island as visited.
- The outer loops ensure all islands are found.

6 Return the final count

- After the full scan, the count represents the number of islands.

```
public int countIslands(char[][] matrix) {
    int[][] visited = new int[matrix.length][matrix[0].length];
    int count=0;
    for (int i=0;i<matrix.length;i++){
        for (int j=0;j<matrix[i].length;j++){
            if (matrix[i][j]=='L' && visited[i][j]==0){
                count +=1;
                traversal(i,j,matrix,visited);
            }
        }
    }
    return count;
}
```

```
class Pair{
    int row, col;
    Pair(int r,int c){
        row=r;
        col=c;
    }
}

void traversal(int i, int j, char[][] matrix, int[][] visited){
    Queue<Pair> q = new LinkedList<>();
    q.add(new Pair(i,j));
    visited[i][j]=1;

    while (!q.isEmpty()){
        Pair curr = q.poll();
        int r = curr.row;
        int c = curr.col;

        for (int nrow=-1; nrow<=1; nrow++){
            for (int ncol = -1; ncol<=1; ncol++){
                int n1 = r + nrow;
                int n2 = c + ncol;

                if(n1>=0 && n1<matrix.length && n2>=0 && n2<matrix[i].length){
                    if(matrix[n1][n2]=='L' && visited[n1][n2]==0){
                        q.add(new Pair(n1,n2));
                        visited[n1][n2]=1;
                    }
                }
            }
        }
    }
}
```

Space: $2O(n*m)$ $\sim O(n*m)$ for visited array & queue.

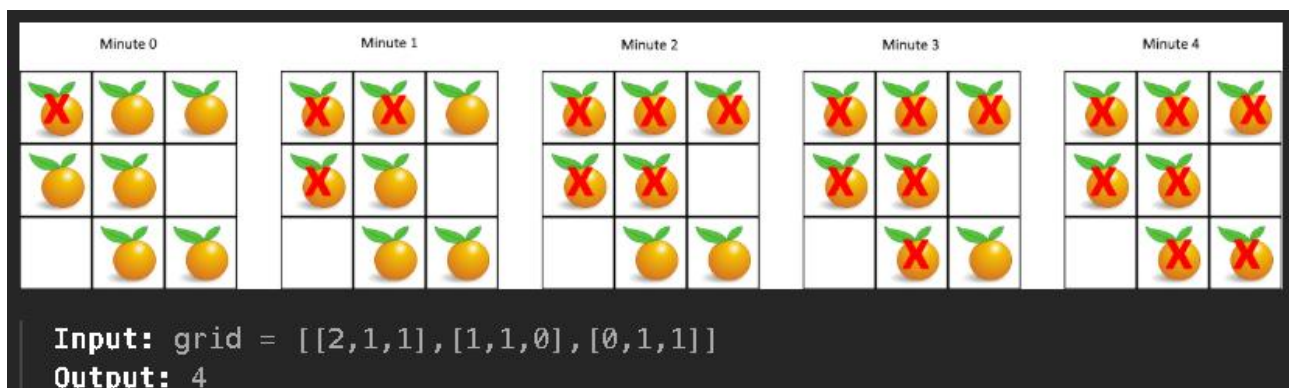
Time: $O(n*m) + 9*O(n*m)$ $O(n*m)$ for matrix traversal & $9O(n*m)$ for bfs traversal assuming every cell of matrix is 'L' & as we are exploring all it's neighbour which will take 9.

You are given an m x n grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.



Optimal:

Intuition

Rotting spreads simultaneously from all rotten oranges, minute by minute, to their 4-directional neighbors.

This is a classic multi-source BFS problem:

- All initially rotten oranges act as starting points.
- In each minute, all oranges that are currently rotten rot their adjacent fresh oranges.
- BFS naturally processes nodes level by level, which perfectly represents time passing.

Each BFS level = 1 minute.

Implementation Steps

1 Add all initial rotten oranges to the queue

- Traverse the grid.
 - Push positions of all cells with value 2 into the queue.
 - These represent rotting sources at **minute 0**.
-

2 Start BFS traversal (minute simulation)

- Initialize minutes = 0.
 - While the queue is not empty:
 - Process all elements currently in the queue (one BFS level).
 - Use a flag to track whether **any fresh orange rots in this minute**.
-

3 Process one BFS level

For each rotten orange in the current level:

- Remove it from the queue.
 - Check its left, right, top, bottom neighbours.
 - If a neighbour is a fresh orange (1):
 - Mark it rotten (2).
 - Add it to the queue.
 - Set the flag to true (rotting occurred this minute).
-

4 Increase time only if rotting happened

- After processing the level:
 - If at least one orange rotted → increment minutes.
 - If nothing rotted → stop BFS (no further spread possible).
-

5 Check for remaining fresh oranges

- Scan the grid after BFS.
- If any fresh orange (1) remains → return -1.

6 Return total time

- If all oranges are rotten → return minutes.

```
static int optimal(int[][] grid) { 1 usage
    Queue<Pair> q = new LinkedList<>();
    for(int r=0;r<grid.length;r++){
        for(int c=0;c<grid[0].length;c++){
            if(grid[r][c]==2) q.add(new Pair(r,c));
        }
    }
    int minutes = 0;
    while(!q.isEmpty()){
        boolean flag = false;
        int s = q.size();
        for(int iteration=0; iteration<s; iteration++){
            Pair curr = q.poll();
            int i = curr.row; int j = curr.col;
            int lcol = j-1;
            int rcol = j+1;
            int trow = i-1;
            int brow = i+1;
            if(lcol != -1 && grid[i][lcol] == 1 ){
                grid[i][lcol] = 2;
                flag = true;
                q.add(new Pair(i,lcol));
            }
            if(rcol != grid[i].length && grid[i][rcol]==1){
                grid[i][rcol] = 2;
                flag = true;
                q.add(new Pair(i,rcol));
            }
        }
        minutes++;
        if(flag == false) break;
    }
    return minutes;
}
```

```

        if(trow != -1 && grid[trow][j]==1){
            grid[trow][j]=2;
            flag=true;
            q.add(new Pair(trow,j));
        }
        if(brow != grid.length && grid[brow][j]==1){
            grid[brow][j]=2;
            flag=true;
            q.add(new Pair(brow,j));
        }
    }
    if(flag) minutes += 1;
    else break;
}
boolean flag = false;
for(int i=0;i<grid.length;i++){
    for(int j=0;j<grid[i].length;j++){
        if(grid[i][j]==1) {
            flag=true;
            break;
        }
    }
}
if(flag) return -1;
return minutes;
}

```

Time: $O(R \times C) + 4 * O(R \times C) + O(R \times C)$

Where: R = number of rows & C = number of columns

Initial scan of grid

- You scan the grid once to push all rotten oranges into the queue: $O(R \times C)$

BFS traversal

- Each rotten cell is added to the queue at most once.
- Each rotten cell's neighbors (up, down, left, right) are checked
→ Total BFS work = $4 * O(R \times C)$

Final scan to check fresh oranges

- Again scans whole grid: $O(R \times C)$

Space: $O(R \times C)$ In worst case, all cells may become rotten and be stored in queue.

You are given an image represented by an $m \times n$ grid of integers image, where $image[i][j]$ represents the pixel value of the image. You are also given three

integers sr , sc , and $color$. Your task is to perform a flood fill on the image starting from the pixel $image[sr][sc]$.

To perform a flood fill:

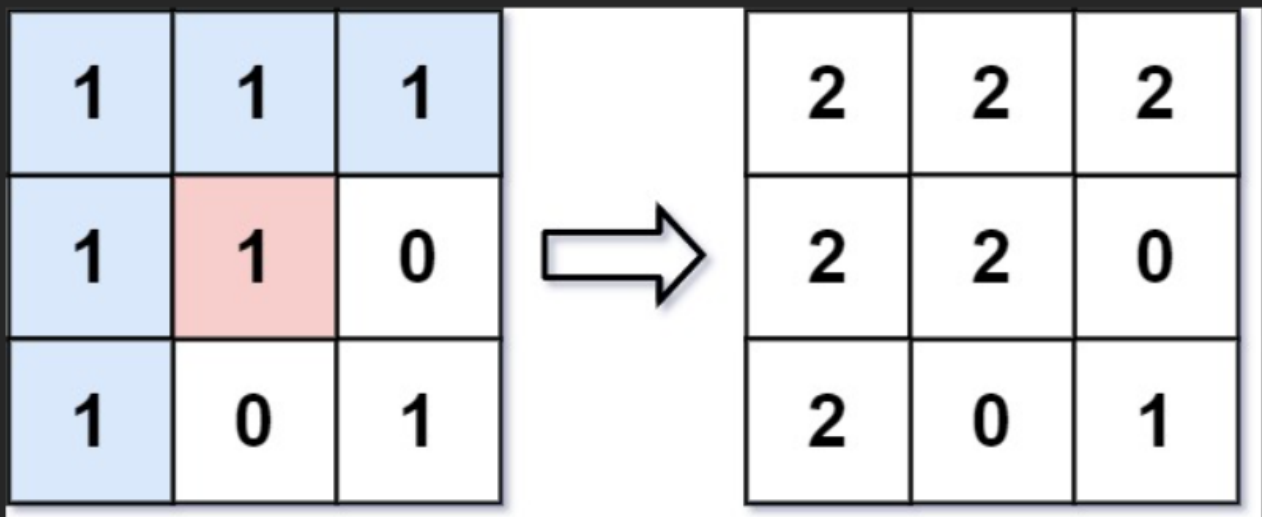
1. Begin with the starting pixel and change its color to $color$.
2. Perform the same process for each pixel that is directly adjacent (pixels that share a side with the original pixel, either horizontally or vertically) and shares the same color as the starting pixel.
3. Keep repeating this process by checking neighboring pixels of the updated pixels and modifying their color if it matches the original color of the starting pixel.
4. The process stops when there are no more adjacent pixels of the original color to update.

Return the modified image after performing the flood fill.

Input: `image = [[1,1,1],[1,1,0],[1,0,1]]`, `sr = 1`, `sc = 1`, `color = 2`

Output: `[[2,2,2],[2,2,0],[2,0,1]]`

Explanation:



From the center of the image with position $(sr, sc) = (1, 1)$ (i.e., the red pixel), all pixels connected by a path of the same color as the starting pixel (i.e., the blue pixels) are colored with the new color.

Note the bottom corner is **not** colored 2, because it is not horizontally or vertically connected to the starting pixel.

★ Intuition

Flood Fill works like spreading paint from a starting pixel.

- Begin from the given starting cell.
- Change its colour.
- Then keep spreading to all adjacent cells (up, down, left, right) that have the same original colour.
- To avoid re-colouring the same cell again and again, maintain a visited matrix.
- Use BFS or DFS so that all reachable cells are processed layer by layer.

The process continues until no adjacent cell of the original colour remains.

```
public int[][] floodFill(int[][] grid, int sr, int sc, int color) {
    int[][] visited = new int[grid.length][grid[0].length];
    Queue<Pair> q = new LinkedList<>();
    int start = grid[sr][sc];
    grid[sr][sc] = color;
    visited[sr][sc] = 1;
    q.add(new Pair(sr,sc));
    while(!q.isEmpty()){
        int s = q.size();
        for(int iter=0; iter<s; iter++){
            Pair curr = q.poll();
            int i = curr.row; int j = curr.col;
            int lcol = j-1; int rcol = j+1;
            int trow = i-1; int brow = i+1;

            if(lcol != -1 && grid[i][lcol] == start && visited[i][lcol]==0){
                grid[i][lcol] = color;
                q.add(new Pair(i,lcol));
                visited[i][lcol]=1;
            }
            if(rcol != grid[i].length && grid[i][rcol]==start && visited[i][rcol]==0 ){
                grid[i][rcol] = color;
                q.add(new Pair(i,rcol));
                visited[i][rcol]=1;
            }
            if(trow != -1 && grid[trow][j]==start && visited[trow][j]==0 ){
                grid[trow][j]=color;
                q.add(new Pair(trow,j));
                visited[trow][j]=1;
            }
            if(brow != grid.length && grid[brow][j]==start && visited[brow][j]==0 ){
                grid[brow][j]=color;
                q.add(new Pair(brow,j));
                visited[brow][j]=1;
            }
        }
    }
    return grid;
}
```

Time Complexity: $4 * O(m \times n)$

- In the worst case, the entire grid has the same color as the starting pixel.
- Each cell:
 - Is added to the queue once
 - Is removed from the queue once
 - Checks at most 4 neighbours (constant work)

Space Complexity: $O(m \times n) + O(m \times n)$

1. Visited matrix $\rightarrow O(m \times n)$
2. Queue: In Worst case holds all cells $\rightarrow O(m \times n)$

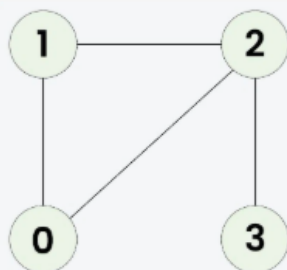
Given an undirected graph with V vertices and E edges, represented as a 2D vector `edges[][]`, where each entry `edges[i] = [u, v]` denotes an edge between vertices u and v , determine whether the graph contains a cycle or not.

Note: The graph can have multiple component.

Input: $V = 4$, $E = 4$, `edges[][] = [[0, 1], [0, 2], [1, 2], [2, 3]]`

Output: true

Explanation:



1 -> 2 -> 0 -> 1 is a cycle.

Intuition: The intuition is that we start from a node, and start doing BFS level-wise, if somewhere down the line, we visit a single node twice, it means we came via two paths to end up at the same node. It implies there is a cycle in the graph because we know that we start from different directions but can arrive at the same node only if the graph is connected or contains a cycle, otherwise we would never come to the same node again.

★ Implementation Steps

1. Build the adjacency list

Convert the edge list into an adjacency list for efficient neighbour traversal.

2. Create a visited array

To track which nodes have already been processed.

3. Traverse each connected component

- For every unvisited node, start a BFS traversal.

4. Initialize BFS with parent tracking

- Push the starting node into the queue with parent -1.
- Mark the starting node as visited.

5. Process nodes using BFS

- Pop a node and its parent from the queue.
- Traverse all adjacent nodes:
 - If the neighbour is unvisited:
 - Mark, it visited.
 - Push it into the queue with current node as its parent.
 - If the neighbour is visited and not equal to the parent:
 - A cycle is detected → return true.

6. Repeat for all components

- If any BFS traversal detects a cycle, stop and return true.

7. Return final result

- If no cycle is found in any component, return false.

```

public boolean isCycle(int nodes, int[][] edges) {
    ArrayList<ArrayList<Integer>> adjList = new ArrayList<>();
    for (int i=0;i<nodes;i++){
        adjList.add(new ArrayList<>());
    }
    for (int i=0;i<edges.length;i++){
        int e1 = edges[i][0];
        int e2 = edges[i][1];

        adjList.get(e1).add(e2);
        adjList.get(e2).add(e1);
    }

    int[] visited = new int[nodes];
    for (int i=0;i<nodes;i++){
        if (visited[i]==0){
            boolean res = traversal(i,adjList,visited);
            if (res) return true;
        }
    }
    return false;
}

```

```

boolean traversal(int i, ArrayList<ArrayList<Integer>> adjList, int[] visited){
    Queue<Pair> q = new LinkedList<>();
    q.add(new Pair(i,-1));
    visited[i] = 1;
    while (!q.isEmpty()){
        Pair p = q.poll();
        int node = p.curr, parent = p.prev;

        for (int e:adjList.get(node)){
            if (visited[e]==0){
                q.add(new Pair(e,node));
                visited[e] = 1;
            }
            else if (visited[e]==1 && e!=parent){
                return true;
            }
        }
    }
    return false;
}

```

Time: $O(V+E) + O(V)+O(V+2E)$

- $O(V+E)$ for creating adjacency list.
- $O(V)$ for connected components.
- $O(V+2E)$ for BFS.

Space: $O(2E) + O(V) + O(V)$

- $O(2E)$ for storing adjacency list.
- $O(V)$ for visited array.

- $O(V)$ for queue.

Here V : number of vertexes & E : number of edges.

Using DFS:

Intuition:

- Perform a **DFS traversal** starting from an unvisited node.
- Mark the current node as **visited**.
- For each adjacent node:
 - If it is **not visited**, recursively explore it.
 - If it is **already visited** and **is not the parent** of the current node, a **cycle exists** because we've reached a previously visited node through a different path.
- If no such case is found during the traversal, then the graph does not contain a cycle.

Using DSU:

Will iterate over all the given edges(u,v), First will check the ultimate parent of both nodes of the given edge. If it is equal means there exist a cycle. Otherwise, will perform union operation.

Given a binary `mat[][]` of $m \times n$, where each cell contains either 0 or 1, find the distance of the nearest 0 for every cell in the grid.

The distance between two cells (i_1, j_1) and (i_2, j_2) is calculated as $|i_1 - i_2| + |j_1 - j_2|$.

You need to return a matrix of the same size, where each cell (i, j) contains the minimum distance from `grid[i][j]` to the nearest cell having value 0.

Note: It is guaranteed that there is at least one cell with value 0 in the grid.

0	0	0
0	1	0
1	1	1

Input: `mat = [[0,0,0],[0,1,0],[1,1,1]]`
Output: `[[0,0,0],[0,1,0],[1,2,1]]`

★ Intuition

For every cell containing 1, we need the distance to the nearest 0.

Instead of running BFS from every 1 (which would be slow), we:

- Start BFS simultaneously from all 0 cells.
- This is called multi-source BFS.

Since BFS expands level by level:

- The first time we reach a 1, we are guaranteed it is reached via the shortest path from the nearest 0.

Thus, the distance assigned during BFS is the minimum distance to a 0.

★ Implementation Steps

1. **Create a visited matrix:** To ensure each cell is processed only once.
2. **Initialize a queue with all 0 cells**
 - Push every cell having value 0 into the queue with distance 0.
 - Mark them as visited.
3. **Define direction arrays:** To move up, right, down, and left from a cell.
4. **Perform BFS**
 - While the queue is not empty:
 - Remove the front cell.

- Store its distance in the matrix.
- Explore its 4 neighbouring cells.

5. Process valid neighbours

For each neighbour:

- Check grid boundaries.
- If the neighbour contains 1 and is unvisited:
 - Push it into the queue with distance current + 1.
 - Mark, it visited.

6. Continue until BFS completes

BFS expands layer by layer, assigning the shortest distance to every cell.

7. Return the updated matrix

Each cell now contains the distance to the nearest 0.


```

class Pair{
    int row,col,dist;
    Pair(int r,int c,int d){
        row=r; col=c; dist=d;
    }
}

public int[][] updateMatrix(int[][] mat) {
    int[][] visited = new int[mat.length][mat[0].length];
    Queue<Pair> q = new LinkedList<>();
    for(int i=0;i<mat.length;i++){
        for(int j=0;j<mat[i].length;j++){
            if(mat[i][j]==0){
                q.add(new Pair(i,j,0));
                visited[i][j] = 1;
            }
        }
    }
    int[] rows = {-1,0,1,0}; int[] cols = {0,1,0,-1};
    while(!q.isEmpty()){
        Pair p = q.poll();
        int r = p.row, c = p.col, d = p.dist;
        mat[r][c] = d;
        for(int i=0;i<4;i++){
            int nrow = r + rows[i];
            int ncol = c + cols[i];
            if(nrow>=0 && nrow<mat.length && ncol>=0 && ncol<mat[r].length){
                if(mat[nrow][ncol]==1 && visited[nrow][ncol]==0){
                    q.add(new Pair(nrow,ncol,d+1));
                    visited[nrow][ncol]=1;
                }
            }
        }
    }
    return mat;
}

```

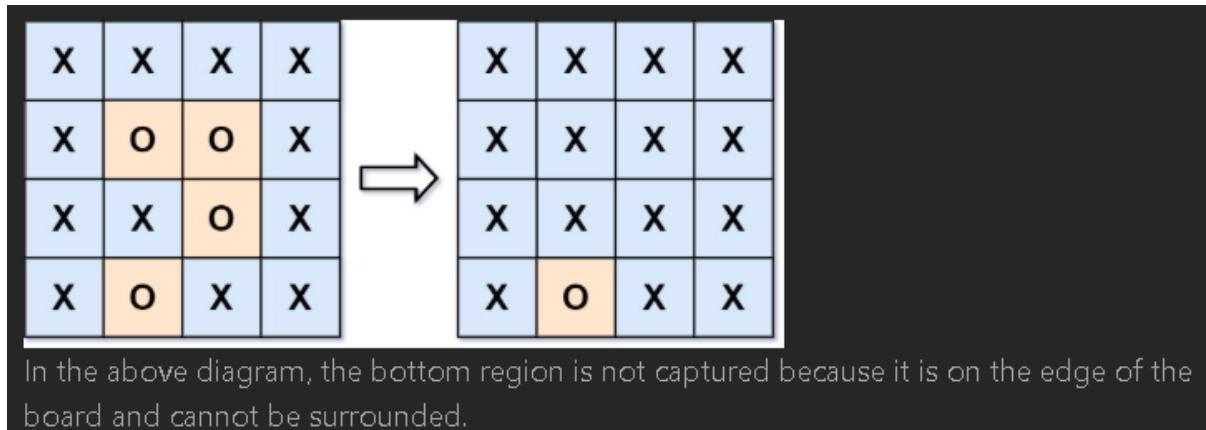
Time: $O(nxm) + O(n \times m) + 4O(nxm)$

- nxm for finding all 0's in the matrix.
- nxm visited matrix initialization.
- Bfs traversal & neighbour check $4O(nxm)$.

Space: $O(nxm) + O(nxm)$

- nxm for visited array.
- nxm for queue.

Given a matrix `mat` of size $N \times M$ where every element is either 'O' or 'X'. Replace all 'O' with 'X' that is surrounded by 'X'. An 'O' (or a set of 'O') is considered to be surrounded by 'X' if there are 'X' at locations just below, just above, just left, and just right of it.



Intuition:

A region of 'O' can only survive if it is connected to the boundary of the matrix. Any 'O' that is completely surrounded by 'X' (i.e., not connected to the border) must be flipped to 'X'.

So instead of trying to directly find surrounded regions (which is tricky), we reverse the thinking:

Mark all 'O' cells that are connected to the boundary as safe. Then flip all remaining 'O' cells to 'X'.

Approach

1. Use DFS (or BFS) starting from boundary cells. Whenever we see an 'O' on the boundary, perform DFS/BFS to mark all connected 'O's as visited (safe).
2. After this traversal, all boundary-connected 'O's remain as they are, because they cannot be surrounded.
3. Traverse the entire matrix:
 - If an 'O' is **visited**, leave it as 'O'.
 - If an 'O' is **not visited**, convert it into 'X'.

```
static void dfs(int row,int col,char[][] mat,int[][] visited,int[] nrows, int[] ncols){
    visited[row][col] = 1;
    for (int i=0;i<4;i++) {
        int r = row + nrows[i];
        int c = col + ncols[i];
        if (r >= 0 && r < mat.length && c >= 0 && c < mat[row].length) {
            if (mat[r][c] == '0' && visited[r][c] == 0)
                dfs(r, c, mat, visited, nrows, ncols);
        }
    }
}
```

```
static void optimal(char[][] mat){ 1 usage
    int[][] visited = new int[mat.length][mat[0].length];
    int[] rows = {0,mat.length-1};
    int[] cols = {0,mat[0].length-1};
    int[] nrows = {-1,0,1,0};
    int[] ncols = {0,1,0,-1};

    for(int r:rows){
        for (int c=0;c<mat[r].length;c++){
            char curr = mat[r][c];
            if(curr == '0' && visited[r][c]==0)
                dfs(r,c,mat,visited,nrows,ncols);
        }
    }
    for(int c:cols){
        for (int r=1;r<mat.length-1;r++){
            char curr = mat[r][c];
            if(curr == '0' && visited[r][c]==0)
                dfs(r,c,mat,visited,nrows,ncols);
        }
    }
    for (int i=0;i<visited.length;i++){
        for (int j=0;j<visited[i].length;j++){
            if(mat[i][j]=='0' && visited[i][j]==0)
                mat[i][j]='X';
        }
    }
}
```

Time Complexity: $O(N \times M)$, since each cell is visited at most once during DFS.

Space Complexity: $O(N \times M)$, due to the visited matrix.

Given a boolean 2D matrix grid of size $n * m$. You have to find the number of distinct islands where a group of connected 1s (horizontally or vertically) forms an island. Two islands are considered to be distinct if and only if one island is not equal to another (not rotated or reflected).

Input:

```
grid[][] = [[1, 1, 0, 1, 1],
            [1, 0, 0, 0, 0],
            [0, 0, 0, 0, 1],
            [1, 1, 0, 1, 1]]
```

Output: 3

Explanation:

```
grid[][] = [[1, 1, 0, 1, 1],
            [1, 0, 0, 0, 0],
            [0, 0, 0, 0, 1],
            [1, 1, 0, 1, 1]]
```

Same colored islands are equal.

We have 4 islands, but 2 of them are equal, So we have 3 distinct islands.

 Intuition:

To count distinct islands, we compare island shapes, not their positions.

For each unvisited land cell, we perform BFS to traverse the entire island. While traversing, we store the relative positions of all island cells with respect to the starting cell (i, j) as (row - i, col - j). This normalizes the island shape, making identical-shaped islands produce the same representation even if they are located at different places.

Each island's normalized shape is stored in a HashSet, which automatically removes duplicates.

Finally, the size of the set gives the number of distinct island shapes.

```
static int optimal(int[][] grid) { 1 usage
    int[][] vis = new int[grid.length][grid[0].length];
    int[] nrows = {0,1,0,-1};
    int[] ncols = {-1,0,1,0};

    Set<ArrayList<ArrayList<Integer>>> s = new HashSet<>();
    for(int i=0;i<grid.length;i++){
        for(int j=0;j<grid[i].length;j++){
            if(vis[i][j]==0 && grid[i][j]==1)
                traversal(i,j,grid,vis,nrows,ncols,s);
        }
    }
    return s.size();
}
```

```

static void traversal(int i, int j, int[][] grid, int[][] vis, int[] nrow, int[] ncol, Set<ArrayList<ArrayList<Integer>>> s){
    Queue<Pair> q = new LinkedList<>();
    ArrayList<ArrayList<Integer>> l = new ArrayList<>();
    q.add(new Pair(i,j));
    l.add(new ArrayList<>(Arrays.asList(0,0)));
    vis[i][j]=1;

    while(!q.isEmpty()){
        Pair curr = q.poll();
        int cr = curr.row;
        int cc = curr.col;

        for(int iter=0; iter<4; iter++){
            int nrow = cr+nrow[iter];
            int ncol = cc+ncol[iter];

            if(nrow>=0 && nrow<grid.length && ncol>=0 && ncol<grid[cr].length){
                if(grid[nrow][ncol]==1 && vis[nrow][ncol]==0){
                    q.add(new Pair(nrow,ncol));
                    vis[nrow][ncol] = 1;
                    l.add(new ArrayList<>(Arrays.asList(nrow-i,ncol-j)));
                }
            }
        }
    }
    s.add(l);
}

```

Time Complexity: $O(n \times m)$

- n = number of rows, m = number of columns.
- Each cell of the grid is visited at most once because of the vis array.
- BFS traversal over all islands together processes every land cell exactly once.
- Storing relative coordinates and inserting into the set is linear in island size, but total work over all islands $\leq n \times m$.

Space Complexity: $O(n \times m)$

- vis array takes $O(n \times m)$ space.
- BFS queue can hold up to $O(n \times m)$ cells in the worst case (one large island).
- The HashSet stores relative coordinate lists whose total size across all islands is also bounded by $O(n \times m)$.

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` $\rightarrow s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_k$ such that:

- Every adjacent pair of words differs by a single letter.
- Every s_i for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- $s_k == \text{endWord}$

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return *the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or 0 if no such sequence exists.*

```
Input: beginWord = "hit", endWord = "cog", wordList =  
["hot","dot","dog","lot","log","cog"]
```

```
Output: 5
```

```
Explanation: One shortest transformation sequence is "hit"  $\rightarrow$  "hot"  
 $\rightarrow$  "dot"  $\rightarrow$  "dog"  $\rightarrow$  "cog", which is 5 words long.
```

Intuition

- Treat **each word as a node** in a graph.
- Two words are connected if they **differ by exactly one character**.
- Every transformation has **equal cost (1 step)**, so we need the **shortest path** from `beginWord` to `endWord`.
- **BFS** is ideal because it explores words **level by level** and guarantees the first time we reach `endWord` is the **minimum number of transformations**.
- A Set is used to **quickly check valid words** and to **mark words as visited** by removing them once used.

Algorithm (Short)

1. If `endWord` is not in `wordList`, return 0.
2. Put all words from `wordList` into a `HashSet` for fast lookup(searching).
3. Initialize a queue and add (`beginWord`, 1) (level = 1).
4. Remove `beginWord` from the set to avoid revisiting (same as visited array).
5. While the queue is not empty:
 - Dequeue a word and its level.

- If it equals endWord, return the level.
 - For each character position:
 - Try replacing it with 'a' to 'z'.
 - If the new word exists in the set:
 - Add it to the queue with level + 1.
 - Remove it from the set.
6. If endWord is never reached, return 0.

```
static int optimal(String beginWord, String endWord, List<String> wordList){
    if(!wordList.contains(endWord)) return 0;

    Set<String> set = new HashSet<>(wordList);
    Queue<Pair> q = new LinkedList<>();
    q.add(new Pair(beginWord, 1));
    set.remove(beginWord);
    int count = 0;
    while (!q.isEmpty()){
        Pair p = q.poll();
        String word = p.s;
        int dist = p.level;
        if(word.equals(endWord)){
            count=dist; break;
        }
        for (int i = 0; i < word.length(); i++) {
            char[] arr = word.toCharArray();
            for (char c = 'a'; c <= 'z'; c++) {
                arr[i] = c;
                String next = new String(arr);
                if (set.contains(next)) {
                    q.add(new Pair(next, dist + 1));
                    set.remove(next);
                }
            }
        }
    }
    return count;
}
```

Time = $O(N \times L \times 26)$

- **BFS traversal:**
 - Each word can generate at most $26 \times L$ possible transformations.
 - Only words in the set are queued, each word visited **once**. $O(N)$

Space Complexity: $O(2N + L)$

- **Queue:** stores words for BFS $\rightarrow O(N)$
- **HashSet:** stores words $\rightarrow O(N)$
- **Temporary char array:** $O(L)$

Where N = number of words in wordlist, L = length of each word

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return *all the shortest transformation sequences from `beginWord` to `endWord`, or an empty list if no such sequence exists. Each sequence should be returned as a list of the words `[beginWord, s1, s2, ..., sk]`.*

```
Input: beginWord = "hit", endWord = "cog", wordList =
["hot","dot","dog","lot","log","cog"]
Output: [["hit","hot","dot","dog","cog"],
["hit","hot","lot","log","cog"]]
Explanation: There are 2 shortest transformation sequences:
"hit" -> "hot" -> "dot" -> "dog" -> "cog"
"hit" -> "hot" -> "lot" -> "log" -> "cog"
```

Approach-1

Intuition

- We need **all shortest transformation sequences**, not just one.
- Since each transformation changes **one character**, the problem is a **shortest path problem in an unweighted graph** $\rightarrow$ BFS.
- Unlike Word Ladder I, we must **store full paths**, because multiple shortest paths may exist.
- To avoid missing valid shortest paths:
 - Words are removed from the dictionary **level by level**, not immediately.
- Once the **shortest path length** is found, we:

- Collect only paths of that length
 - Ignore longer paths
-

Algorithm

1. Put all words of wordList into a HashSet for O(1) lookup.
2. Use a BFS queue that stores **paths** (List<String>).
3. Start BFS with a path containing only beginWord.
4. For each BFS level:
 - Maintain a temporary set usedWords for words used in this level.
 - For each path in the current level:
 - If last word equals endWord, add path to result.
 - Otherwise, generate all valid one-letter transformations:
 - If the new word exists in the dictionary:
 - Add it to usedWords
 - Create a new path by copying current path and enqueue it.
 - After finishing the level, remove all usedWords from the dictionary.
5. Return all collected shortest paths.

```

static List<List<String>> optimal(String beginWord, String endWord, List<String> wordList){
    List<List<String>> result = new ArrayList<>();
    if(!wordList.contains(endWord)) return result ;
    Set<String> set = new HashSet<>(wordList);
    Queue<List<String>> q = new LinkedList<>();
    q.add(new ArrayList<>(List.of(beginWord)));
    set.remove(beginWord);
    while (!q.isEmpty()){
        int s = q.size();
        Set<String> usedWords = new HashSet<>();
        for (int i=0;i<s;i++){
            List<String> curr = q.poll();
            String lastWord = curr.getLast();
            if (lastWord.equals(endWord)){
                result.add(curr);
                continue;
            }
            char[] arr = lastWord.toCharArray();
            for (int j=0;j<lastWord.length();j++) {
                char original = arr[j];
                for (char c = 'a'; c <= 'z'; c++) {
                    arr[j] = c;
                    String newWord = new String(arr);
                    if (set.contains(newWord)) {
                        usedWords.add(newWord);
                        List<String> newPath = new ArrayList<>(curr);
                        newPath.add(newWord);
                        q.add(newPath);
                    }
                }
                arr[j] = original;
            }
        }
        for (String e: usedWords) {
            set.remove(e);
        }
    }
    return result;
}

```

Time Complexity = $O((N \times L) + (P \times L))$

- **Neighbour generation:**
 - For each word, try changing every character (L positions $\times$ 26 letters).
 - 26 is constant $\Rightarrow O(L)$ per word.
- **BFS traversal:**
 - Each word is visited only once because it is removed from the set after its BFS level.
 - Total cost $\Rightarrow O(N \times L)$.
- **Path copying:**
 - Every valid transformation creates a new path using: `new ArrayList<>(curr)`
 - Copying a path takes $O(\text{path length})$.

- If P paths are generated, total cost $\Rightarrow O(P \times L)$.

Space Complexity = $O(N \times (P+L))$

- **HashSet:** Stores all dictionary words $\Rightarrow O(N)$.
- **Queue:** Stores complete paths during BFS.
 - If P paths are in the queue and each has average length L , space $\Rightarrow O(P \times L)$.
- **Result:** Stores all shortest paths, which is included in $O(P \times L)$.

Approach-2

Think of each word as a **node in a graph**, where an edge exists between two words if they differ by **only one character**.

- **BFS** is used first because it always finds the **shortest distance** from beginWord to every reachable word.
- While doing BFS, we store the **level (distance)** of each word from beginWord.
- Once we know these levels, we don't need to explore all possible paths again.
- We start from endWord and **move backwards** only to words whose level is **exactly one less** than the current word.
- Since the level decreases by 1 at every step, we are guaranteed to follow **only the shortest transformation sequences**.
- Finally, we reverse each path because we built it from endWord to beginWord.

Time Complexity

Let:

- N = number of words
- L = length of each word
- **BFS:** $O(N \times L \times 26) \approx O(NL)$
- **DFS:** In the worst case, visits every shortest transformation sequence.
Complexity = $O(P \times L)$, where P is the number of shortest paths.

Overall: $O(N \times L \times 26 + P \times L)$

Space Complexity

- HashSet: $O(N)$
- HashMap: $O(N)$
- Queue: $O(N)$
- DFS recursion + current path: $O(L)$
- Output storage: O (total size of all shortest paths)

Auxiliary Space: $O(N + L)$ (excluding the output).

Bipartite graph:

Formal Definition

A graph $G(V, E)$ is bipartite if:

- V can be split into two sets U and V
- Every edge connects a vertex in U to a vertex in V
- There is no edge between vertices of the same set

Coloring View

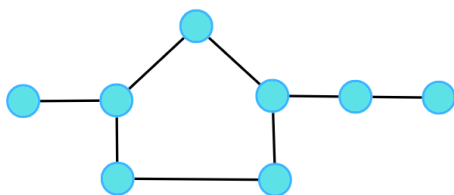
If you can color the graph using 2 colors (say Red & Blue) such that adjacent nodes have different colors then the graph is bipartite.

Note:

- Any graph with an odd-length cycle is NOT bipartite
- Any graph with an even-length cycle is always bipartite.
- Any graph having no-cycle will always be bipartite.

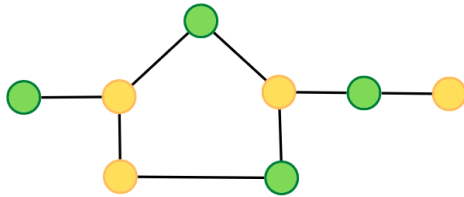
Given an adjacency list of a graph adj of V no. of vertices having 0 based index. Graph may also contain multiple components. Check whether the graph is bipartite or not.

Input:



Output: false

Explanation:



Intuition:

The intuition is the brute force of filling colours using any traversal technique, just make sure no two adjacent nodes have the same colour. If at any moment of traversal, we find the adjacent nodes to have the same colour, it means that there is an odd cycle, or it cannot be a bipartite graph.

Algorithm:

- Use two colours (0 and 1).
- Start from an un-coloured node and assign it a color.
- Traverse the graph using BFS / DFS.
- If a neighbour is uncolored, assign it the opposite color of the current node.
- If a neighbour already has the same color, the graph is not bipartite.
- Repeat this process for all disconnected components of the graph.
- If no conflicts are found in any component, the graph is bipartite.

```

static boolean bfs(int node, int[][] adjList, int[] coloured){
    Queue<Integer> q = new LinkedList<>();
    q.add(node);
    coloured[node] = 0;
    while (!q.isEmpty()){
        int curr = q.poll();
        for (int e: adjList[curr]){
            if (coloured[e]==-1){
                q.add(e);
                coloured[e] = (coloured[curr]==0) ? 1 : 0;
            }
            else if (coloured[e]==coloured[curr]) return false;
        }
    }
    return true;
}

static boolean optimal1(int[][] adjList){ 1 usage
    int[] coloured = new int[adjList.length];
    Arrays.fill(coloured, -1); //tc = O(n)

    for (int i=0;i< coloured.length;i++){
        if (coloured[i]==-1) {
            if (!bfs(i,adjList,coloured)) return false;
        }
    }
    return true;
}

```

```

static boolean dfs(int node,int color,int[][] adjList,int[] coloured){
    coloured[node]=color;
    for (int e: adjList[node]){
        if (coloured[e]==-1){
            int tempColor = color==0 ? 1 : 0;
            if (!dfs(e,tempColor,adjList,coloured)) return false;
        }
        else if (coloured[e]==color) return false;
    }
    return true;
}

static boolean optimal2(int[][] adjList){ 1 usage
    int[] coloured = new int[adjList.length];
    Arrays.fill(coloured, -1); //tc = O(n)
    for (int i=0;i< coloured.length;i++){
        if (coloured[i]==-1) {
            if (!dfs(i, color: 0,adjList,coloured))
                return false;
        }
    }
    return true;
}

```

Time: $O(V) + O(V) + O(V+2E)$

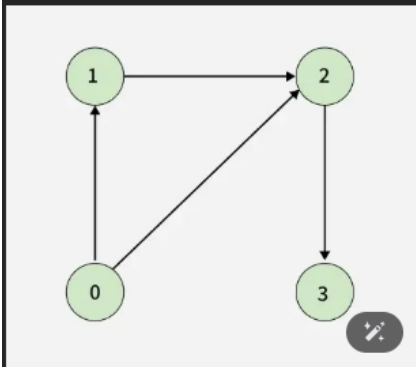
- Initializing and filling the coloured array takes $O(V)$ time.
- $O(V)$ for connected components
- $O(V+2E)$ for traversal.

Space: $O(2V)$

- $O(V)$ for coloured array.
- The BFS queue can hold up to all vertices in the worst case $\rightarrow O(V)$

Given a directed graph with V vertices and E edges, represented as a 2D vector `edges[][]`, where each entry `edges[i] = [u, v]` denotes an edge between vertices u and v , determine whether the graph contains a cycle or not. The graph can have multiple components.

Input: $V = 4$, `edges[][] = [[0, 1], [0, 2], [1, 2], [2, 3]]`



Output: false

Explanation: no cycle in the graph

Using DFS:

During DFS, a cycle exists if we reach a node that is already in the current DFS path.

Each node can be in three states:

1. **0 $\rightarrow$ Unvisited:** Node has not been visited yet.
2. **1 $\rightarrow$ Visiting (in current DFS path):** Node is currently in recursion stack.
3. **2 $\rightarrow$ Visited (DFS completed):** Node and all its descendants are fully processed.

Cycle Detection Rule

- If during DFS we reach a node that is already in state 1, it means we found a back edge, which confirms a cycle.
-

Algorithm

1. Create an adjacency list from the given edges.
2. Initialize a visited array with all nodes marked as **unvisited (0)**.
3. Traverse all nodes:
 - If a node is unvisited, start DFS from it.
4. During DFS:
 - Mark the current node as visiting (1).
 - For every adjacent node:
 - If it is unvisited, recursively call DFS if at some point we are getting true for any particular node means we have detected one cycle & no need to explore other node will return true for all the above function calls.
 - If it is already in visiting state (1), a cycle is detected return true.
 - After all neighbours are processed, mark the node as visited (2) & return false.
5. If a cycle is detected at any point, return true.
6. If all nodes are processed without detecting a cycle, return false.


```

boolean DFS(int node, ArrayList<ArrayList<Integer>> adjList, int[] visited){
    visited[node] = 1;
    for (int e: adjList.get(node)){
        if (visited[e]==0){
            if (DFS(e, adjList, visited)) return true;
        }
        else if (visited[e]==1) return true;
    }
    visited[node] = 2;
    return false;
}

public boolean isCyclic(int nodes, int[][] edges) {
    ArrayList<ArrayList<Integer>> adjList = new ArrayList<>();
    for (int i=0;i<nodes;i++){
        adjList.add(new ArrayList<>());
    }
    for (int i=0;i<edges.length;i++){
        int e1 = edges[i][0];
        int e2 = edges[i][1];
        adjList.get(e1).add(e2);
    }

    int[] visited = new int[nodes];
    for (int i=0;i<nodes;i++){
        if (visited[i]==0){
            if (DFS(i,adjList,visited))
                return true;
        }
    }
    return false;
}

```

Time: $O(V+E) + O(N) + O(V+E)$ V= no of vertices & E=no of edges.

- $O(V+E)$ for creating adjacency list.
- $O(N)$ for components of graph.
- $O(V+E)$ for dfs traversal.

Space: $O(E) + O(V) + O(V)$

- $O(E)$ for storing adjList.
- $O(V)$ for visited[] & another $O(V)$ for stack-space.

Using Topological sort:

As we know we will only get to linear ordering of nodes if the graph does not contains any cycle also it is only applicable for the directed graph so will use the same concept here & if we will get the linear ordering means there is no cycle so will return false otherwise will return true.

There is a directed graph of n nodes with each node labeled from 0 to $n - 1$. The graph is represented by a 0-indexed 2D integer array `graph` where `graph[i]` is an integer array of nodes adjacent to node i , meaning there is an edge from node i to each node in `graph[i]`.

A node is a terminal node if there are no outgoing edges. A node is a safe node if every possible path starting from that node leads to a terminal node (or another safe node).

Return *an array containing all the safe nodes of the graph*. The answer should be sorted in ascending order.

Input: `graph = [[1,2],[2,3],[5],[0],[5],[],[[]]]`

Output: `[2,4,5,6]`

Explanation: The given graph is shown above. Nodes 5 and 6 are terminal nodes as there are no outgoing edges from either of them. Every path starting at nodes 2, 4, 5, and 6 all lead to either node 5 or 6.

DFS Approach

- The graph is directed and may contain cycles.
- A node is safe if every path starting from it ends at a terminal node (having no outgoing edges).
- Any node that is part of a cycle, or can reach a cycle, is not safe.
- Cycles are the only reason a path does not end at a terminal node.
- So, the problem reduces to detecting cycles in an undirected graph and marking all nodes involved in or leading to cycles as unsafe.

```

static boolean DFS(int node, int[][] adjList, int[] visited){
    visited[node] = 1;
    for (int e: adjList[node]){
        if (visited[e]==0){
            if (DFS(e, adjList, visited)) return true;
        }
        else if (visited[e]==1) return true;
    }
    visited[node] = 2;
    return false;
}

static List<Integer> optimal(int[][] adjList){ 1 usage
    int[] visited = new int[adjList.length];
    for (int i=0;i< adjList.length;i++){
        if (visited[i]==0){
            DFS(i,adjList,visited);
        }
    }
    List<Integer> res = new ArrayList<>();
    for (int i=0;i<visited.length;i++){
        if (visited[i]==2)
            res.add(i);
    }
    return res;
}

```

Time: $O(N) + O(V+E) + O(V)$, V= no of vertices & E=no of edges.

- $O(N)$ for components of graph.
- $O(V+E)$ for dfs traversal.
- $O(V)$ for finding safe node.

Space: $O(V) + O(V) + O(V)$

- $O(V)$ for safe nodes.
- $O(V)$ for visited[] & another $O(V)$ for stack-space.

Using Topological sort:

First will reverse the connection between nodes & after that will simply do form a linear ordering of nodes & the nodes will we will get in this order will be the safe nodes.

We will reverse the edges between two nodes... because without this we cannot perform the traversal on whole graph using topo-sort.

Topo-Sort Problems

Topological Sort:

- It is a linear ordering of vertices in a directed acyclic graph (DAG) such that: For every directed edge $u \rightarrow v$, node u appears before v in the ordering.
- There can be more than one valid topological order
- It represents dependency order.

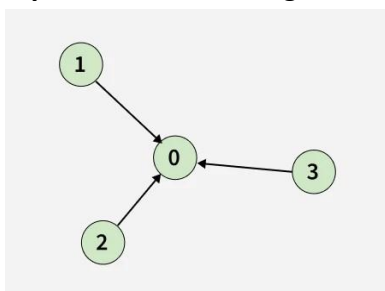
Topological sort is only possible on a Directed Acyclic Graph (DAG) because:

- It requires every directed edge $u \rightarrow v$ to have u before v .
- A cycle creates circular dependencies (e.g., $A \rightarrow B \rightarrow C \rightarrow A$), making such an ordering impossible.
- Every DAG has at least one vertex with in-degree 0, allowing algorithms like Kahn's Algorithm to construct a valid ordering.
- Therefore, a graph has a topological ordering if and only if it is a DAG.

Given a Directed Acyclic Graph (DAG) of V (0 to $V-1$) vertices and E edges represented as a 2D list of `edges[i]`, where each entry `edges[i] = [u, v]` denotes a directed edge $u \rightarrow v$. Return the topological sort for the given graph.

Note: As there are multiple Topological orders possible, you may return any of them.

Input: $V = 4$, $E = 3$, `edges[i] = [[3, 0], [1, 0], [2, 0]]`



Output: true

Explanation: The output true denotes that the order is valid. Few valid Topological orders for the given graph are:

[3, 2, 1, 0]

[1, 2, 3, 0]

[2, 3, 1, 0]

Using DFS

Core Idea

- Use DFS to go as deep as possible before fixing the position of a node.
 - A node is added to the order only after all its outgoing neighbors are processed.
 - This ensures that for every edge $u \rightarrow v$, node u comes before v in the final ordering.
-

Why Stack Is Used

- During DFS, once a node finishes (all its children are done), it is pushed onto a stack.
- This means dependencies are pushed first, and parents are pushed later.
- Popping the stack gives the correct topological order.
- The stack contains nodes in reverse topological order.
- Popping all elements from the stack produces a valid topological sorting.

```
static void dfs(int node, ArrayList<ArrayList<Integer>> adjList, int[] vis, Stack<Integer> s){
    vis[node] = 1;
    for(int e: adjList.get(node)){
        if(vis[e]==0)
            dfs(e,adjList,vis,s);
    }
    s.push(node);
}

static ArrayList<Integer> optimal(int V, int[][] edges) { 1 usage
    ArrayList<ArrayList<Integer>> adjList = new ArrayList<>();
    for(int i=0;i<V;i++){
        adjList.add(new ArrayList<>());
    }
    for(int i=0;i<edges.length;i++){
        adjList.get(edges[i][0]).add(edges[i][1]);
    }
    int[] vis = new int[V];
    Stack<Integer> s = new Stack<>();
    for(int i=0;i<V;i++){
        if(vis[i]==0)
            dfs(i,adjList,vis,s);
    }
    ArrayList<Integer> res = new ArrayList<>();
    while(!s.isEmpty()){
        res.add(s.pop());
    }
    return res;
}
```

Time: $O(V+E)$

- Building the adjacency list takes $O(V + E)$ time.
- DFS visits each vertex **once** $\rightarrow O(V)$.
- During DFS, every directed edge is explored **once** $\rightarrow O(E)$.
- Pushing and popping nodes from the stack takes $O(V)$.

Space: $O(V+E)$

- Adjacency list storage $\rightarrow O(V + E)$
- Visited array $\rightarrow O(V)$
- Stack (for topo order) $\rightarrow O(V)$
- Recursive call stack (worst case) $\rightarrow O(V)$

Using BFS (kahn's Algorithm)

This method works by keeping track of the in-degree of each vertex, which represents the number of incoming edges to that vertex. Any vertex with an in-degree of zero has no prerequisites and can therefore safely appear first in the topological order.

Short Intuition:

- In topological sorting, a node can be processed only when it has no remaining dependencies.
- In-degree represents how many prerequisites a node has.
- All nodes with in-degree 0 are added to a queue first.
- Using BFS, nodes are removed one by one and added to the result.
- Removing a node reduces the in-degree of its neighbours.
- If a neighbour's in-degree becomes 0, it is added to the queue.
- This process continues until all possible nodes are processed.

```

public ArrayList<Integer> topoSort(int V, int[][] edges) {
    ArrayList<ArrayList<Integer>> adjList = new ArrayList<>();
    int[] inDegree = new int[V];
    for(int i=0;i<V;i++){
        adjList.add(new ArrayList<>());
    }
    for(int i=0;i<edges.length;i++){
        int vStart = edges[i][0];
        int vEnd = edges[i][1];
        adjList.get(vStart).add(vEnd);
        inDegree[vEnd] += 1;
    }

    int[] vis = new int[V];
    ArrayList<Integer> res = new ArrayList<>();
    Queue<Integer> q = new LinkedList<>();
    for (int i=0;i<inDegree.length;i++){
        if(inDegree[i]==0){
            q.add(i);
            vis[i]=1;
        }
    }
    while (!q.isEmpty()){
        int curr = q.poll();
        res.add(curr);
        for (int e: adjList.get(curr)){
            inDegree[e] -= 1;
            if (inDegree[e]==0){
                q.add(e);
                vis[e] = 1;
            }
        }
    }
    return res;
}

```

Time Complexity = $O(V + E)$

- Building the adjacency list and in-degree array: $O(V + E)$
- Each vertex is added to and removed from the queue **once** → $O(V)$
- Each directed edge is processed once when reducing in-degree → $O(E)$

Space: $O(V+E)$

- Adjacency list storage: $O(V + E)$
- In-degree array: $O(V)$
- Queue: $O(V)$
- Result list: $O(V)$

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]

Output: false

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Approach-1: By detecting cycle in an un-directed graph

Each course → a **node**. Each prerequisite [a, b] → a **directed edge b → a**. In this method we will check all the components of either using BFS or DFS to detect cycle if we find any cycle means will not finish all the courses so will return false, if there is no cycle will return true.

Approach-2: Using topological sort

If we are getting the topological order & we no we only get the order if the graph does not contain any cycle in this case will return true... otherwise will return false.

A new alien language uses the English alphabet, but the order of letters is unknown. You are given a list of words[] from the alien language's dictionary, where the words are claimed to be sorted lexicographically according to the language's rules.

Your task is to determine the correct order of letters in this alien language based on the given words. If the order is valid, return a string containing the unique letters in lexicographically increasing order as per the new language's rules. If there are multiple valid orders, return any one of them.

However, if the given arrangement of words is inconsistent with any possible letter ordering, return an empty string ("").

A string a is lexicographically smaller than a string b if, at the first position where they differ, the character in a appears earlier in the alien language than the corresponding

character in b. If all characters in the shorter word match the beginning of the longer word, the shorter word is considered smaller.

Input: words[] = ["caa", "aaa", "aab"]

Output: true

Explanation: A possible correct order of letters in the alien dictionary is "cab".

The pair "caa" and "aaa" suggests 'c' appears before 'a'.

The pair "aaa" and "aab" suggests 'a' appear before 'b' in the alien dictionary.

So, 'c' $\rightarrow$ 'a' $\rightarrow$ 'b' is a valid ordering.

Intuition

The given words are already sorted according to an **unknown (alien) alphabet order**.

- By comparing **adjacent words**, the **first differing characters** tell us the **relative order** of letters.
- These order relations form a **directed graph** ($u \rightarrow v$ means u comes before v).
- If this graph has a **cycle** or we do a comparison on two words & don't find a different character means, no valid letter order exists.
- Otherwise, a **topological order** of this graph gives a valid alien alphabet.

Algorithm

Mark all **unique characters** that appear in the words.

1. **Build a graph:**
 - For each adjacent word pair:
 - If word1 is longer and starts with word2 $\rightarrow$ return "" (invalid).
 - Find the first differing character and add a directed edge.
 - Update indegree.
2. **Topological Sort (Kahn's Algorithm):**
 - Push all characters with indegree = 0 into a queue. Along with that will also count total number of unique characters.
 - Repeatedly pop, add to result, and reduce indegree of neighbours.
3. **Cycle check:**
 - If the result length $\neq$ number of unique characters $\rightarrow$ return "".

4. Return the constructed order.

```
public String findOrder(String[] words) {
    ArrayList<ArrayList<Integer>> adList = new ArrayList<>();
    int[] inDegree = new int[26];
    int[] present = new int[26];
    for (int i=0;i<26;i++){
        adList.add(new ArrayList<>());
    }
    for (String w : words) {
        for (char c : w.toCharArray()) {
            present[c - 'a'] = 1;
        }
    }

    for(int i=0;i< words.length-1;i++){
        String w1 = words[i], w2 = words[i+1];
        if (w1.length() > w2.length() && w1.startsWith(w2)) {
            return "";
        }
        int l = Math.min(w1.length(), w2.length());
        for (int j=0;j<l;j++){
            if (w1.charAt(j) != w2.charAt(j)){
                int u = w1.charAt(j) - 'a';
                int v = w2.charAt(j) - 'a';
                adList.get(u).add(v);
                inDegree[v] += 1;
                break;
            }
        }
    }
}
```

```
Queue<Integer> q = new LinkedList<>();
int totalChars = 0;
for (int i=0;i<26;i++){
    if (present[i]==1){
        totalChars += 1;
        if (inDegree[i]==0) q.add(i);
    }
}
StringBuilder sb = new StringBuilder();
while (!q.isEmpty()){
    int curr = q.poll();
    sb.append((char) (curr+97));
    for (int nei: adList.get(curr)){
        inDegree[nei] -= 1;
        if (inDegree[nei]==0) q.add(nei);
    }
}
if (sb.length() != totalChars) return "";
return sb.toString();
```

Time: $O(N \times L + K + E)$

- Marking present characters: $O(N \times L)$
- Building the graph:
 - Compare adjacent words

- At most L character comparisons per pair
- $O(N \times L)$
- **Topological Sort (Kahn's Algorithm)**
 - Each character processed once $\rightarrow O(K)$
 - Each edge processed once $\rightarrow O(E)$ where $E \leq N$ (at most one edge per word pair)
 - $O(K + E)$

Space Complexity: $O(K+E)$

- Adjacency List: Stores at most K nodes and E edges: $O(K + E)$
- Arrays (inDegree, present, queue): $O(3K)$

Where N = number of words, L =length of words, K =no of nodes, E = no. of edges

Shortest Path Algorithms & problems

Given a Directed Acyclic Graph of V vertices from 0 to $n-1$ and a 2D Integer array(or vector) `edges[ ][ ]` of length E , where there is a directed edge from `edge[i][0]` to `edge[i][1]` with a distance of `edge[i][2]` for all i .

Find the shortest path from `src(0)` vertex to all the vertices and if it is impossible to reach any vertex, then return `-1` for that vertex.

```
Input: V = 4, E = 2, edges = [[0,1,2], [0,2,1]]
Output: [0, 2, 1, -1]
Explanation: Shortest path from 0 to 1 is 0->1 with
edge weight 2. Shortest path from 0 to 2 is 0->2
with edge weight 1. There is no way we can reach 3,
so it's -1 for 3.
```

Intuition (Why TopoLogical Sort here?)

Because the graph is a **DAG (no cycles)**, we can arrange its nodes in a **topological order** such that every node appears **before all nodes it points to**.

Processing nodes in this order guarantees that:

- When we compute the shortest distance for a node,
- All paths leading to it have already been processed.

This allows us to compute shortest paths in **one pass**, without revisiting nodes, making topological sort the **most efficient and correct choice** for shortest path in a DAG.

Approach:

- Build the adjacency list from the given edges.
- Perform topological sort of the graph using DFS and store nodes in a stack.
- Initialize distance array with infinity; set $\text{dist}[\text{src}] = 0$.
- Process nodes in topological order:
 - For each node, relax all outgoing edges if the node is reachable means update it's distance if we can reach in lower cost that the cost stored inside it..
- Mark unreachable nodes by replacing infinity with -1.
- Return the distance array.

Time Complexity: $O(V+E)$

1. Building adjacency list: $O(E)$
2. Topological sort (DFS): $O(V+E)$
3. Relaxing edges in topo order: $O(V+E)$

Space Complexity: $O(V+E)$

1. Adjacency list: $O(V + E)$
2. Visited array: $O(V)$
3. Stack for topo sort: $O(V)$
4. Distance array: $O(V)$

You are given an undirected graph with V vertices numbered from 0 to $V-1$ and E edges, represented as a 2D array $\text{edges}[][]$, where each element $\text{edges}[i] = [u, v]$ represents an undirected edge between vertices u and v .

Your task is to find the shortest path distance from a given source vertex src to all other vertices in the graph.

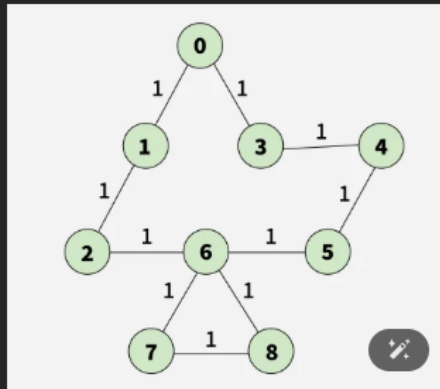
If a vertex is not reachable from the source, return -1 for that vertex.

Note: All edges have unit weight (1).

Input: $V = 9$, $E = 10$,

$\text{edges}[][] = [[0, 1], [0, 3], [1, 2], [3, 4], [4, 5], [2, 6], [5, 6], [6, 7], [6, 8], [7, 8]]$, $\text{src} = 0$

Output: $[0, 1, 2, 1, 2, 3, 3, 4, 4]$



Approach:

We can solve this problem by simply doing a BFS traversal.

```
public int[] shortestPath(int V, int[][] edges, int src) {
    ArrayList<ArrayList<Integer>> adj = new ArrayList<>();
    for(int i=0;i<V;i++){
        adj.add(new ArrayList<>());
    }
    for(int i=0;i<edges.length;i++){
        int u = edges[i][0];
        int v = edges[i][1];
        adj.get(u).add(v);
        adj.get(v).add(u);
    }

    int[] vis = new int[V];
    int[] dist = new int[V];
    Arrays.fill(dist,-1);
    dist[src] = 0;

    Queue<Integer> q = new LinkedList<>();
    q.add(src);
    vis[src] = 1;

    while(!q.isEmpty()){
        int curr = q.poll();
        for(int v: adj.get(curr)){
            if(vis[v]==0){
                q.add(v);
                vis[v] = 1;
                dist[v] = dist[curr] + 1;
            }
        }
    }
    return dist;
}
```

Given an undirected, weighted graph with V vertices numbered from 0 to $V-1$ and E edges, represented by 2d array `edges[][]`, where `edges[i]=[u, v, w]` represents the edge between the nodes u and v having w edge weight.

You have to find the shortest distance of all the vertices from the source vertex `src`, and return an array of integers where the i th element denotes the shortest distance between i th node and source vertex `src`.

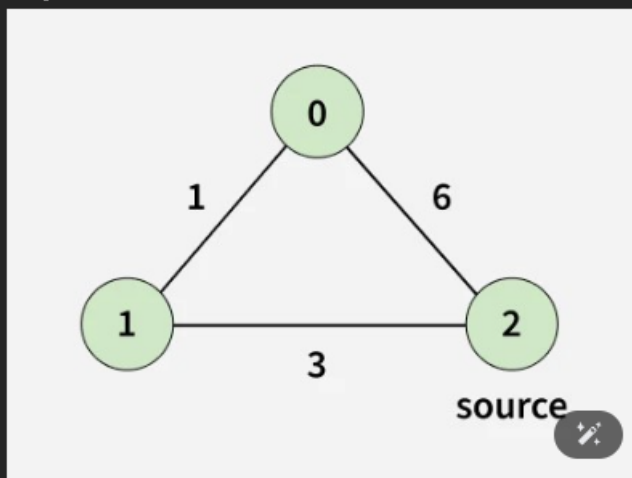
Note: The Graph is connected and doesn't contain any negative weight edge.

It is guaranteed that all the shortest distance will fit in a 32-bit integer.

Input: $V = 3$, `edges[][] = [[0, 1, 1], [1, 2, 3], [0, 2, 6]]`, `src = 2`

Output: `[4, 3, 0]`

Explanation:



Shortest Paths:

For 2 to 0 minimum distance will be 4. By following path $2 \rightarrow 1 \rightarrow 0$

For 2 to 1 minimum distance will be 3. By following path $2 \rightarrow 1$

For 2 to 2 minimum distance will be 0. By following path $2 \rightarrow 2$

Approach-1: Using Queue

We can solve it by simply doing a bfs traversal by storing (node,distance) in the queue. As we know in bfs we explore all the path so there might be a case where we explore the path through which we cannot find our goal. For that we use Priority-Queue to choose min path distance at each step.

Time = $O(VE)$

Approach-2: Using Priority Queue (Min-heap)

We are given a connected, undirected, weighted graph with non-negative edge weights. The goal is to find the shortest distance from a source node `src` to every other node.

Key idea behind Dijkstra

If all edge weights are non-negative, then once we reach a node via the smallest possible distance, that distance will never improve later.

So, we follow this greedy strategy:

1. Start from the source with distance 0
2. Always pick the node that currently has the minimum known distance
3. Try to improve the distances of its neighbours
4. Repeat until all nodes are processed

To always get the minimum-distance node efficiently, we use a Priority-Queue (**min-heap**) ordered by distance.

```
static class Pair{
    int node;
    int dist;
    Pair(int n, int d){
        node = n;
        dist = d;
    }
}
```

```

static int[] optimal1(int V, int[][] edges, int src){
    List<List<Pair>> adj = new ArrayList<>();
    for (int i=0;i<V;i++){
        adj.add(new ArrayList<>());
    }
    for (int i=0;i<edges.length;i++){
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];
        adj.get(u).add(new Pair(v,w));
        adj.get(v).add(new Pair(u,w));
    }

    int[] dist = new int[V];
    Arrays.fill(dist,Integer.MAX_VALUE);
    PriorityQueue<Pair> q = new PriorityQueue<>((a, b) -> a.dist - b.dist);
    q.add(new Pair(src,0));
    dist[src] = 0;
    while (!q.isEmpty()){
        Pair curr = q.poll();
        int vertex = curr.node;
        int weight = curr.dist;
        for (Pair p: adj.get(vertex)){
            int neiNode = p.node;
            int neiDist = p.dist;
            if (dist[neiNode] > weight+neiDist){
                dist[neiNode] = weight+neiDist;
                q.add(new Pair(neiNode,dist[neiNode]));
            }
        }
    }
    return dist;
}

```

Time: $O(V+E) + O(V) + O(V \times (\log(\text{heap size}) + (V-1) \times \log(\text{heap size})))$

$O(E \log V)$

- For building adjacency List (V+E).
- $O(V)$ for initially assigning initially distance for each node as +infinity.
- No of times while loop will execute = $O(V)$
 - $\log(\text{heap size})$ for removing element from the Priority Queue.
heap size = $V \times V \Rightarrow E$ (Assuming every node is connected with all other nodes)
 - $O(V-1)$ for number of neighbour nodes.
 - ❖ $\log(\text{heap size})$ for adding element in the queue.

Space: $O(2 \times E) + O(V) + O(E)$

- $O(2E)$ for adjacency list.
- $O(V)$ for dist[].
- $O(E)$ for priority queue.

Where E= number of edges, V=number of vertices.

Approach-3: Using TreeSet

- TreeSet keeps elements **sorted**, so:
 - pollFirst() always gives the node with **minimum distance**.
- When a shorter path to a neighbour is found:
 - Remove the old (node, distance) from the TreeSet if it's not dis[node] is not infinite. In this way it reduces number of iteration in some cases but with that it will also take logarithmic time to delete it from the Tree-Set. So overall it will not decrease the complexity compare to previous approach.
 - Update dist [neighbour].
 - Insert new (neighbour, updatedDist) into TreeSet.

Time: $O(E \log V)$

Space: $O(V+E)$

Where E= number of edges, V=number of vertices.

```
static int[] optimal2(int v, int[][] edges, int src){
    List<List<Pair>> adj = new ArrayList<>();
    for (int i=0;i<v;i++){
        adj.add(new ArrayList<>());
    }
    for (int i=0;i<edges.length;i++){
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];
        adj.get(u).add(new Pair(v,w));
        adj.get(v).add(new Pair(u,w));
    }
    int[] dist = new int[v];
    Arrays.fill(dist,Integer.MAX_VALUE);

    TreeSet<Pair> ts = new TreeSet<>((a, b) -> {
        if (a.dist != b.dist)
            return a.dist - b.dist;
        return a.node - b.node;
    });
    ts.add(new Pair(src,0));
    dist[src] = 0;
    while (!ts.isEmpty()){
        Pair curr = ts.pollFirst();
        int vertex = curr.node;
        int weight = curr.dist;
        for (Pair p: adj.get(vertex)){
            int neiNode = p.node;
            int neiDist = p.dist;
            if (dist[neiNode] > weight+neiDist){
                if (dist[neiNode] != Integer.MAX_VALUE){
                    ts.remove(new Pair(neiNode,dist[neiNode]));
                }
                dist[neiNode] = weight+neiDist;
                ts.add(new Pair(neiNode,dist[neiNode]));
            }
        }
    }
    return dist;
}
```

You are given a weighted undirected graph with n vertices numbered from 1 to n and m edges along with their weights. Find the shortest path between vertex 1 and vertex n . Each edge is given as $\{a, b, w\}$, denoting an edge between vertices a and b with weight w .

If a path exists, return a list of integers where the first element is the total weight of the shortest path, and the remaining elements are the nodes along that path (from 1 to n). If no path exists, return a list containing only $\{-1\}$.

Input: $n = 5, m = 6, \text{edges} = [[1, 2, 2], [2, 5, 5], [2, 3, 4], [1, 4, 1], [4, 3, 3], [3, 5, 1]]$

Output: 5

Explanation: Shortest path from 1 to n is by the path 1 4 3 5 whose weight is 5.

Approach

- Since the graph is weighted and all weights are non-negative, we use Dijkstra's algorithm.
- Always expand the node with the minimum current distance using a priority queue.
- Keep updating shorter distances to neighbouring nodes.
- Store the parent of each node to reconstruct the shortest path.
- If the destination node is unreachable, return -1; otherwise, build the path using the parent array.

Given an $n \times n$ binary matrix grid, return the length of the shortest clear path in the matrix.

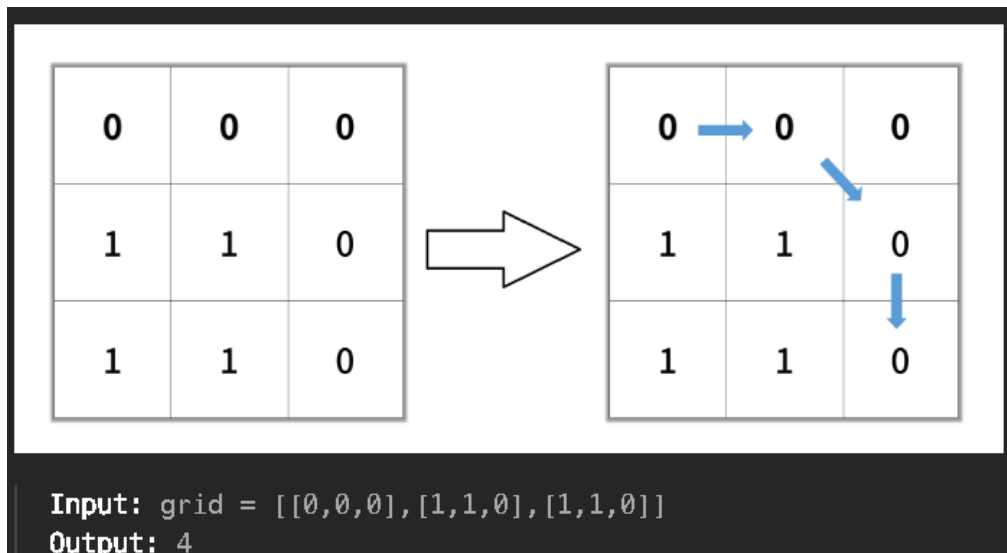
If there is no clear path, return -1.

A clear path in a binary matrix is a path from the top-left cell (i.e., $(0, 0)$) to the bottom-right cell (i.e., $(n - 1, n - 1)$) such that:

All the visited cells of the path are 0.

All the adjacent cells of the path are 8-directionally connected (i.e., they are different and they share an edge or a corner).

The length of a clear path is the number of visited cells of this path.



Approach:

- The grid is binary:
0 → free cell, 1 → blocked cell.
- We need the shortest path from (0,0) to (n-1,n-1).
- Movement is allowed in 8 directions (horizontal, vertical, diagonal).
- Since each move has equal cost (1), the problem becomes an unweighted shortest path problem.
- BFS is the optimal choice because it explores cells level by level, guaranteeing the shortest distance.
- A distance matrix is used to avoid revisiting cells with a longer path.

Algorithm

1. If the start or end cell is blocked, return -1.
2. Initialize a distance matrix with ∞ .
3. Push the starting cell (0,0) with distance 1 into a queue.
4. While the queue is not empty:
 - Dequeue the current cell.
 - Explore all 8 neighboring cells.
 - For each valid and unblocked neighbor:

- If a shorter distance is found, update distance and push into the queue.
- If the destination is reached, return its distance.

5. If the destination is never reached, return -1.

Complexity

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(n^2)$

You are a hiker preparing for an upcoming hike. You are given heights, a 2D array of size rows x columns, where heights[row][col] represents the height of cell (row, col). You are situated in the top-left cell, (0, 0), and you hope to travel to the bottom-right cell, (rows-1, columns-1) (i.e., 0-indexed). You can move up, down, left, or right, and you wish to find a route that requires the minimum effort.

A route's effort is the maximum absolute difference in heights between two consecutive cells of the route.

Return the minimum effort required to travel from the top-left cell to the bottom-right cell.

Input: heights = [[1,2,2],[3,8,2],[5,3,5]]

Output: 2

Explanation: The route of [1,3,5,3,5] has a maximum absolute difference of 2 in consecutive cells.

This is better than the route of [1,2,2,2,5], where the maximum absolute difference is 3.

Approach:

- Each move between adjacent cells has an effort equal to the absolute height difference.
- The total path effort is the maximum edge effort along that path.
- We want a path from (0,0) to (n-1,m-1) such that this maximum value is minimized.
- This problem fits Dijkstra's algorithm, where:

- Nodes → grid cells
 - Edge weight → height difference
 - Path cost → maximum edge weight so far
 - A min-heap ensures we always expand the cell with the least current effort.
-

Algorithm

1. Initialize a diffArr matrix with ∞ to store minimum effort to reach each cell.
 2. Use a priority queue ordered by minimum effort.
 3. Start from (0,0) with effort 0.
 4. While the priority queue is not empty:
 - Extract the cell with the smallest effort.
 - If it is the destination, return the effort.
 - For each of the 4 adjacent cells:
 - Compute $\text{newEffort} = \max(\text{currentEffort}, \text{heightDifference})$.
 - If newEffort is smaller than stored value, update and push into the queue.
 5. Return the effort when the destination is reached.
-

```

public static int optimal(int[][] matrix) {
    int n = matrix.length; int m = matrix[0].length;
    int[][] diffArr = new int[n][m];
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            diffArr[i][j] = Integer.MAX_VALUE;
        }
    }
    int[] rows = {0,-1,0,1}; int[] columns = {-1,0,1,0};
    PriorityQueue<Pair> pq = new PriorityQueue<>((a,b) -> a.diff-b.diff);
    pq.add(new Pair(0,0,0));
    diffArr[0][0] = 0;
    while(!pq.isEmpty()){
        Pair p = pq.poll();
        int currRow = p.row;
        int currCol = p.col;
        int currDiff = p.diff;
        if(currRow==n-1 && currCol==m-1){
            return currDiff;
        }
        for(int i=0;i<4;i++){
            int neiRow = rows[i] + currRow; int neiCol = columns[i] + currCol;
            if(neiRow>=0 && neiRow<n && neiCol>=0 && neiCol<m){
                int diff = Math.abs(matrix[neiRow][neiCol] - matrix[currRow][currCol]);
                int newEffort = Math.max(diff, currDiff);
                if (newEffort < diffArr[neiRow][neiCol]) {
                    diffArr[neiRow][neiCol] = newEffort;
                    pq.add(new Pair(neiRow, neiCol, newEffort));
                }
            }
        }
    }
    return 0;
}

```

Complexity

- Time: $O(n*m \log(n*m))$
- Space: $O(n*m)$

There are n cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return *the cheapest price from `src` to `dst` with at most `k` stops*. If there is no such route, return `-1`.

Input: $n = 4$, $flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]$, $src = 0$, $dst = 3$, $k = 1$

Output: 700

Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost $100 + 600 = 700$.

Note that the path through cities $[0,1,2,3]$ is cheaper but is invalid because it uses 2 stops.

Approach:

This problem is a shortest path problem with a constraint:

- We want the minimum cost from src to dst
- But we are allowed at most k stops (i.e., at most $k + 1$ edges)

A normal Dijkstra's algorithm fails here because:

- It always picks the cheapest path without considering the number of stops
- A cheaper path with more than k stops is invalid

So instead, we use a BFS-like traversal (level by level by stops) while:

- Tracking the current city
- Tracking the total cost
- Tracking the number of stops used so far

Algorithm:

1. Create an adjacency list from the given flights array.
2. Initialize a $prices[]$ array with ∞ and set $prices[src] = 0$.
3. Use a queue to perform BFS, storing (currentNode, totalCost, stopsUsed).
4. Push ($src, 0, -1$) into the queue.
5. While the queue is not empty:
 - Dequeue an element.

- If node is destination skip it. We might reach the destination at min price. Skip it.
 - If stopsUsed + 1 > k, skip further expansion.
 - For each neighbor:
 - If a cheaper cost is found within k stops, update prices[] and enqueue the new state.
6. If prices[dst] is still ∞ , return -1; otherwise, return prices[dst].

Given start, end and an array arr of n numbers. At each step, start is multiplied with any number in the array and then mod operation with 100000 is done to get the new start. Your task is to find the minimum steps in which end can be achieved starting from start. If it is not possible to reach end, then return -1.

Why can't we use DP?

DP works when a state depends on **already computed states** (like dp[i] depends on dp[i-1], dp[i-2]).

Here, if we define:

dp[x] = minimum operations to reach x

there is **no fixed way** to compute dp[x] because:

- We don't know all the previous numbers that can reach x.
- Due to the modulo operation, states can form **cycles** (e.g., $a \rightarrow b \rightarrow c \rightarrow a$), so there is no order in which we can fill the DP table.

Hence, this is **not a DP problem** but a **shortest path problem on a graph**.

Intuition Behind the BFS Solution

- Each number from 0 to 999 is a node.
- From a node x, we can go to:

$$(x \times arr[i]) \bmod 1000$$

- Every multiplication takes exactly 1 operation, so every edge has equal weight (1).

- When all edges have the same weight, BFS gives the shortest path.
- BFS explores level by level:
 - Level 0: start
 - Level 1: all numbers reachable in 1 operation
 - Level 2: all numbers reachable in 2 operations
 - and so on...
- Therefore, the first time we reach end, we have found the minimum number of operations.

Role of vis[]

In your code:

- $\text{vis}[x] = -1 \rightarrow x$ has not been visited.
- $\text{vis}[x] = k \rightarrow$ minimum steps required to reach x is k .
- Once a node is visited, we do not process it again because BFS already reached it using the shortest path.

Complexity

- Total states = 1000 (0 to 999).
- For each state, we try all elements of arr.
- Time Complexity: $O(1000 \times \text{arr.length})$.
- Space Complexity: $O(1000)$ for the visited array and queue.

You are given a network of n nodes, labeled from 1 to n . You are also given times, a list of travel times as directed edges $\text{times}[i] = (u_i, v_i, w_i)$, where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target.

We will send a signal from a given node k . Return *the minimum time it takes for all the n nodes to receive the signal*. If it is impossible for all the n nodes to receive the signal, return -1.

Approach:

We apply Dijkstra's algorithm to find the shortest time from the source to all nodes, and the maximum of these times gives the network delay.

Algorithm

- Build an adjacency list from the given edges.
- Initialize all distances as infinity and set distance of source k to 0.
- Use a min-priority queue to pick the node with the smallest current time.
- Relax all outgoing edges of the extracted node.
- Repeat until the priority queue is empty.
- If any node is unreachable, return -1; otherwise return the maximum distance.

You are in a city that consists of n intersections numbered from 0 to $n - 1$ with bi-directional roads between some intersections. The inputs are generated such that you can reach any intersection from any other intersection and that there is at most one road between any two intersections.

You are given an integer n and a 2D integer array `roads` where `roads[i] = [ui, vi, timei]` means that there is a road between intersections u_i and v_i that takes $time_i$ minutes to travel. You want to know in how many ways you can travel from intersection 0 to intersection $n - 1$ in the shortest amount of time.

Return the number of ways you can arrive at your destination in the shortest amount of time. Since the answer may be large, return it modulo $10^9 + 7$.

```
Input: n = 7, roads = [[0,6,7],[0,1,2],[1,2,3],[1,3,3],
[6,3,3],[3,5,1],[6,5,1],[2,5,1],[0,4,5],[4,6,2]]
Output: 4
Explanation: The shortest amount of time it takes to go
from intersection 0 to intersection 6 is 7 minutes.
The four ways to get there in 7 minutes are:
- 0 → 6
- 0 → 4 → 6
- 0 → 1 → 2 → 5 → 6
- 0 → 1 → 3 → 5 → 6
```

Approach:

The city intersections form a connected, undirected weighted graph. We need to find two things at the same time:

1. The shortest time to reach every intersection from 0
2. The number of distinct ways to reach each intersection using that shortest time

Since all road times are positive, Dijkstra's algorithm is ideal.

While finding the shortest distances, we also count how many shortest paths lead to each node.

Algorithm

1. Build an undirected adjacency list from the given roads.
2. Initialize distance array with infinity and ways array with 0.
3. Use a min-priority queue for Dijkstra's algorithm.
4. Set, $\text{distance}[0] = 0$ and $\text{ways}[0] = 1$.
5. While the priority queue is not empty:
 - Extract the node with minimum distance.
 - Relax its edges:
 - If a shorter path is found, update distance and copy ways.
 - If an equal shortest path is found, add ways $(\text{mod } 10^9 + 7)$.
6. Return $\text{ways}[n-1]$ (number of shortest paths to destination).

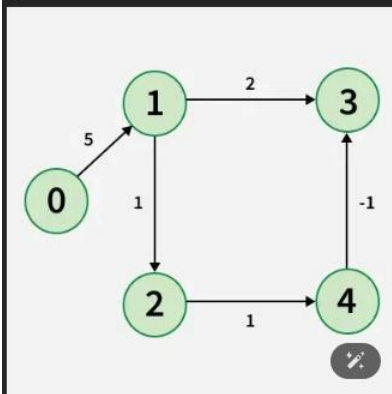
Bellman-Ford Algorithm:

- It is used to find shortest distance from given source to all other nodes in a directed graph. Unlike Dijkstra it is also applicable for graph having negative weight.
- Dijkstra is applicable for both directed & un-directed graph having no negative edge weight & negative weight cycle (Cycle whose edge weight sum is -ve).
- It helps to detect negative weight cycles in directed graph.
- To use bellman-ford with un-directed graph (having no negative weight) we need to convert it in the form of directed graph by adding parallel edges between nodes.

Given an weighted graph with V vertices numbered from 0 to $V-1$ and E edges, represented by a 2d array $\text{edges}[i][j]$, where $\text{edges}[i] = [u, v, w]$ represents a direct edge from node u to v having w edge weight. You are also given a source vertex src .

Your task is to compute the shortest distances from the source to all other vertices. If a vertex is unreachable from the source, its distance should be marked as 10^8 . Additionally, if the graph contains a negative weight cycle, return $[-1]$ to indicate that shortest paths cannot be reliably computed.

Input: $V = 5$, $edges[][] = [[1, 3, 2], [4, 3, -1], [2, 4, 1], [1, 2, 1], [0, 1, 5]]$, $src = 0$



Output: [0, 5, 6, 6, 7]

Approach:

- Go through all edges multiple times to update the shortest distances. After repeating this process for the number of nodes - 1, the shortest paths are found.
- Relaxation:** If taking a certain edge gives a shorter path to a point, update that point with the new shorter distance.
- Why Repeat N-1 Times?**
Since in a graph of N nodes, in worst case, you will take N-1 edges to reach from the first to the last, thereby we **relax all edges N-1 times**.
- After relaxation: If any edge can still be relaxed $\rightarrow$ a **negative weight cycle exists**

Algorithm

- Initialize distance array with ∞ (large value) for all vertices
- Set $distance[src] = 0$
- Repeat **V-1 times**:
 - For every edge ($u \rightarrow v$, weight w)
 - If $distance[u] + w < distance[v]$, update $distance[v]$
- Perform **one extra iteration** over all edges:
 - If any distance still reduces $\rightarrow$ **negative cycle detected**
 - Return $\{-1\}$
- Otherwise, return the distance array

```

public static int[] optimal(int V, int[][] edges, int src) {
    int[] distances = new int[V];
    Arrays.fill(distances, (int)1e8);
    distances[src] = 0;

    for(int i=0; i<V; i++){
        for(int j=0; j<edges.length; j++){
            int u = edges[j][0];
            int v = edges[j][1];
            int w = edges[j][2];

            if(distances[u] != 1e8 && distances[u]+w < distances[v]){
                distances[v] = distances[u]+w;
            }
        }
    }

    for(int j=0; j<edges.length; j++){
        int u = edges[j][0];
        int v = edges[j][1];
        int w = edges[j][2];

        if(distances[u] != 1e8 && distances[u]+w < distances[v]){
            return new int[] {-1};
        }
    }

    return distances;
}

```

Time Complexity: $O(V \cdot E)$, where V = no. of vertices and E = no. of Edges.

Space Complexity: $O(V)$ for the distance array which stores the minimized distances.

Floyd–Warshall Algorithm:

It is a dynamic programming algorithm used to find the shortest paths between **all pairs of vertices** in a weighted graph, including graphs with **negative edge weights** (but no negative cycles).

You are given a weighted directed graph, represented by an adjacency matrix, `dist[][]` of size $n \times n$, where `dist[i][j]` represents the weight of the edge from node *i* to node *j*. If there is no direct edge, `dist[i][j]` is set to a large value (i.e., 10^8) to represent infinity.

The graph may contain negative edge weights, but it does not contain any negative weight cycles.

Your task is to find the shortest distance between every pair of nodes *i* and *j* in the graph.

Note: Modify the distances for every pair in place.

Input: `dist[][] = [[0, 4, 108, 5, 108], [108, 0, 1, 108, 6], [2, 108, 0, 3, 108], [108, 108, 1, 0, 2], [1, 108, 108, 4, 0]]`

Output: `[[0, 4, 5, 5, 7], [3, 0, 1, 4, 6], [2, 6, 0, 3, 5], [3, 7, 1, 0, 2], [1, 5, 5, 4, 0]]`

Approach:

For every pair (i, j), check whether going through an intermediate node k gives a shorter path than the direct one.

Algorithm

1. Treat the given matrix `dist[][]` as the initial distance between all node pairs.
2. For each node `k` from 0 to `n-1`, consider it as an intermediate node.
3. For every pair of nodes (i, j):
 - If both $i \rightarrow k$ and $k \rightarrow j$ are reachable,
 - Update `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])`.
4. After all iterations, `dist[i][j]` stores the shortest distance between every pair of nodes.

Note: To check whether given graph contains negative weight or cycle or not for that will check the diagonal element distance & if it is negative for any node will return there is a negative weight cycle.

```
public static void optimal(int[][] dist) {
    for(int iter=0;iter<dist.length;iter++){
        for(int i=0;i<dist.length;i++){
            for(int j=0;j<dist.length;j++){
                int u = dist[i][iter];
                int v = dist[iter][j];
                if( (u != (int)1e8 && v!=(int)1e8 )){
                    dist[i][j] = Math.min(dist[i][j],u+v);
                }
            }
        }
    }
}
```

Time Complexity: $O(V^3)$, where `V` is the number of vertices in the graph. This is because we have three nested loops iterating over the vertices.

Space Complexity: $O(V^2)$, where `V` is the number of vertices in the graph. This is due to the storage of the distance matrix, which is of size `V x V`.

Minimum Spanning Tree & Disjoint Set

A **Spanning tree** is tree (no cycle) having exactly 'N' node & 'N-1' edges & all nodes are reachable from each other. A given graph can have more than one spanning tree.

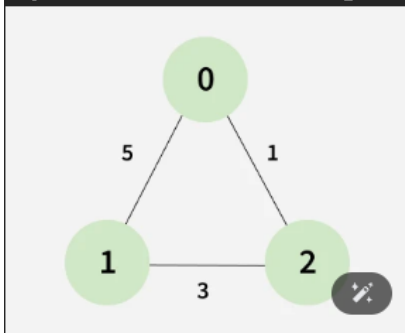
A **MST** is a sub-graph(S) of Graph(G) having minimum total edges weight & does not contain any cycles.

Ways for finding MST:

1. Kruskal's algorithm
2. Prim's algorithm

Given a weighted, undirected, and connected graph with V vertices and E edges, your task is to find the sum of the weights of the edges in the Minimum Spanning Tree (MST) of the graph. The graph is provided as a list of edges, where each edge is represented as [u, v, w], indicating an edge between vertex u and vertex v with edge weight w.

Input: V = 3, E = 3, Edges = [[0, 1, 5], [1, 2, 3], [0, 2, 1]]



Output: 4

Approach:

This solution uses **Prim's Algorithm** with a **Priority Queue**:

The intuition of this algorithm is the greedy technique used for every node. If we carefully observe, for every node, we are greedily selecting its unvisited adjacent node with the minimum edge weight (as the priority queue here is a min-heap and the topmost element is the node with the minimum edge weight). Doing so for every node, we can get the sum of all the edge weights of the minimum spanning tree and the spanning tree itself (if we wish to) as well.

Algorithm

1. Build an **adjacency list** from the edge list.
2. Initialize:

- visited[] array to track included nodes
 - Min-heap (Priority-Queue) to pick minimum edge
3. Push (0, 0) into the priority queue (start node with cost 0).
 4. While the priority queue is not empty:
 - Extract the node with the **smallest edge weight**
 - If not visited:
 - Mark it visited
 - Add its weight to minSum
 - Push all unvisited neighbors into the priority queue
 5. Return minSum as the MST weight.

```

public int spanningTree(int n, int[][] edges) {
    List<List<Pair>> adj = new ArrayList<>();
    for (int i=0;i<n;i++){
        adj.add(new ArrayList<>());
    }
    for (int i=0;i<edges.length;i++){
        int u = edges[i][0]; int v = edges[i][1]; int w = edges[i][2];
        adj.get(u).add(new Pair(v,w)); adj.get(v).add(new Pair(u,w));
    }

    int minSum = 0;
    int[] vis = new int[n];
    PriorityQueue<Pair> pq = new PriorityQueue<>((a,b)->a.dist-b.dist);
    pq.add(new Pair(0,0));
    while (!pq.isEmpty()){
        Pair p = pq.poll();
        int currNode = p.node;
        int currDis = p.dist;
        if (vis[currNode]==0) {
            vis[currNode] = 1;
            minSum += currDis;
            for (Pair nei: adj.get(currNode)){
                int neiNode = nei.node;
                int neiDist = nei.dist;
                if(vis[neiNode]==0){
                    pq.add(new Pair(neiNode,neiDist));
                }
            }
        }
    }
    return minSum;
}

```

Time Complexity: $O(E \cdot \log E) + O(E \cdot \log E) \sim O(E \cdot \log E)$, where E = no. of given edges.

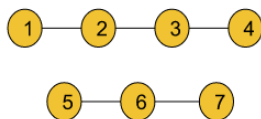
The maximum size of the priority queue can be E so after at most E iterations the priority

queue will be empty and the loop will end. Inside the loop, there is a pop operation that will take $\log E$ time. This will result in the first $O(E \cdot \log E)$ time complexity. Now, inside that loop, for every node, we need to traverse all its adjacent nodes where the number of nodes can be at most E . If we find any node unvisited, we will perform a push operation and for that, we need a $\log E$ time complexity. So, this will result in the second $O(E \cdot \log E)$.

Space Complexity: $O(E) + O(V)$, where E = no. of edges and V = no. of vertices. $O(E)$ occurs due to the size of the priority queue and $O(V)$ due to the visited array. If we wish to get the mst, we need an extra $O(V-1)$ space to store the edges of the most.

Disjoint-Set: T.C: $O(4 \alpha)$

Question: Given two components of an undirected graph.



Here, question is whether node 1 and node 5 are in the same component or not.

Approach:

- Now, in order to solve this question, we can use either the [DFS](#) or [BFS](#) traversal technique like if we traverse the components of the graph, we can find that node 1 and node 5 are not in the same component. This is actually the **brute force** approach whose time complexity is $O(N+E)$ (N = no. of nodes, E = no. of edges). But **using a Disjoint Set data structure we can solve this same problem in $O(1)$** .
- The disjoint Set data structure is generally used for **dynamic graphs**.

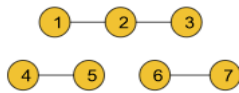
Dynamic graph:

A dynamic graph generally refers to a graph that keeps on changing its configuration. Let's deep dive into it using an example:

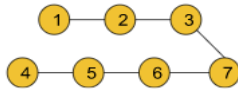
- Let's consider the edge information for the given graph as: $\{\{1,2\}, \{2,3\}, \{4,5\}, \{6,7\}, \{5,6\}, \{3,7\}\}$. Now if we start adding the edges one by one, in each step the structure of the graph will change. So, after each step, if we perform the same operation on the graph while updating the edges, the result might be different. In this case, the graph will be considered a dynamic graph.
- For example, after adding the first 4 edges if we look at the graph, we will find that node 4 and node 1 belong to different components but after adding all 6 edges if we search for the same, we will figure out that node 4 and node 1 belong to the same component.

{1,2}
 {2,3}
 {4,5}
 {6,7}
 {5,6}
 {3,7}

After adding the edges upto {6,7}



After adding all the edges



- So, after any step, if we try to figure out whether two arbitrary nodes u and v belong to the same component or not, Disjoint Set will be able to answer this query in constant time.

Functionalities of Disjoint Set data structure:

The disjoint set data structure generally provides two types of functionalities:

- Finding the ultimate parent for a particular node (*findPar()*)
- Union (in broad terms this method basically adds an edge between two nodes)
 - Union by rank
 - Union by size

Union by rank:

```

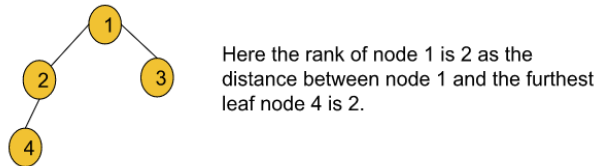
static class DisjointSet { 2 usages  Ⓐ ADITYA PATEL *
    int[] rank; 4 usages
    int[] parent; 8 usages
    DisjointSet(int n) { 1 usage  Ⓐ ADITYA PATEL *
        rank = new int[n];
        parent = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }
    int findUltimateParent(int node) { 5 usages  Ⓐ ADITYA PATEL *
        if (node == parent[node]) return node;
        return parent[node] = findUltimateParent(parent[node]);
    }

    void unionByRank(int u, int v) { 5 usages  Ⓐ ADITYA PATEL *
        int ultimateU = findUltimateParent(u); int ultimateV = findUltimateParent(v);
        if (ultimateU == ultimateV) return;

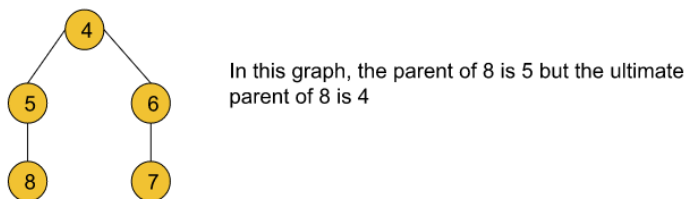
        int rankU = rank[ultimateU];
        int rankV = rank[ultimateV];
        if (rankU < rankV) parent[ultimateU] = ultimateV;
        else if (rankV < rankU) parent[ultimateV] = ultimateU;
        else {
            parent[ultimateV] = ultimateU;
            rank[ultimateU] = rankU + 1;
        }
    }
}
  
```

Before discussing Union by rank we need to discuss some terminologies:

Rank: The rank of a node generally refers to the distance (the number of nodes including the leaf node) between the furthest leaf node and the current node. Basically, rank includes all the nodes beneath the current node.



Ultimate parent: ultimate parent refers to the root node.



Now let's discuss the implementation of the union by rank function. In order to implement Union by rank, we basically need two arrays of size N (no. of nodes). One is the **rank** and the other one is the **parent**. The rank array basically stores the rank of each node and the parent array stores the ultimate parent for each node.

rank array: This array is initialized with zero.

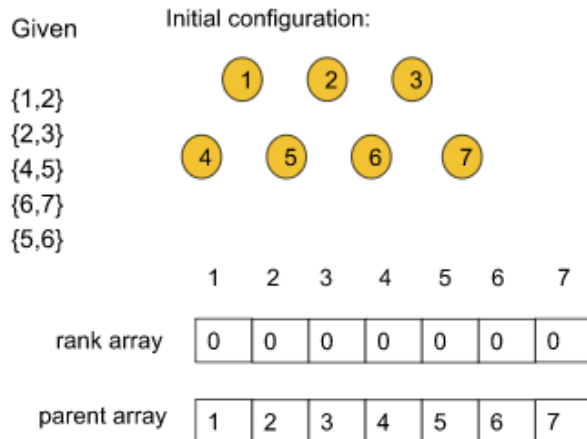
parent array: The array is initialized with the value of nodes i.e. $\text{parent}[i] = i$.

Algorithm:

1. Firstly, the Union function requires two nodes (**let's say u and v**) as arguments. Then we will find the ultimate parent (using the `findPar()` function that is discussed later) of u and v . Let's consider the ultimate parent of u is **pu** and the ultimate parent of v is **pv** .
2. After that, we will find the rank of **pu** and **pv** .
3. Finally, we will connect the ultimate parent with a smaller rank to the other ultimate parent with a larger rank. But if the ranks are equal, we can connect any parent to the other parent and we will increase the rank by one for the parent node to whom we have connected the other one.

Let's understand it further using the below example.

Given the edges of a graph are: $\{\{1,2\}, \{2,3\}, \{4,5\}, \{6,7\}, \{5,6\}\}$



Observation 1:

If we carefully observe, we are only concerned about the ultimate parent but not the immediate parent.

Let's see ***why we need to find the ultimate parents.***

- After union by rank operations, if we are asked (refer to the above picture) if node 5 and node 7 belong to the same component or not, the answer must be yes. If we carefully look at their immediate parents, they are not the same but if we consider their ultimate parents they are the same i.e. node 4. So, we can determine the answer by considering the ultimate parent. That is why we need to find the ultimate parent.

So, here comes the **findPar() function** which will help us to find the ultimate parent for a particular node.

findPar() function: This function actually takes a single node as an argument and finds the ultimate parent for each node.

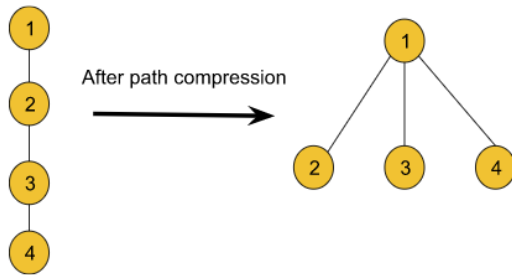
Observation 2:

Now, if we try to find the ultimate parent (typically using recursion) of each query separately, it will end up taking $O(\log N)$ time complexity for each case. But we want the operation to be done in a constant time. This is where the ***path compression technique*** comes in.

Using the ***path compression technique***, we can reduce the time complexity nearly to constant time.

What is path compression?

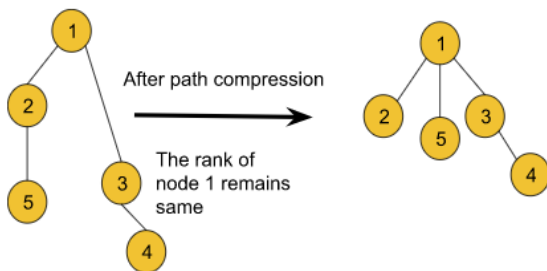
Basically, connecting each node in a particular path to its ultimate parent refers to path compression. Let's understand it using the following illustration:



How the time complexity reduces:

- Before path compression, if we had tried to find the ultimate parent for node 4, we had to traverse all the way back to node 1 which is basically the height of size $\log N$. But after path compression, we can easily access the ultimate parent with a single step. Thus, the traversal reduces and as a result the time complexity also reduces.

Though using the path compression technique, it seems like the rank of the node is also changing, we cannot be sure about it. So, we will not make any changes to the rank array while applying path compression. The following example depicts an example:



Note: We cannot change the ranks while applying path compression.

Overall, findPar() method helps to reduce the time complexity of the **union by the rank** method as it can find the ultimate parent within constant time.

Algorithm:

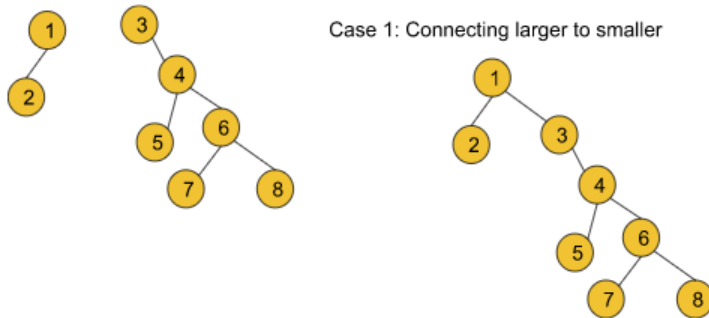
This process is done using the backtracking method.

1. **Base case:** If the node and the parent of the node become the same, it will return the node.
2. We will call the findPar() function for a node until it hits the base case and while backtracking, we will update the parent of the current node with the returned value.

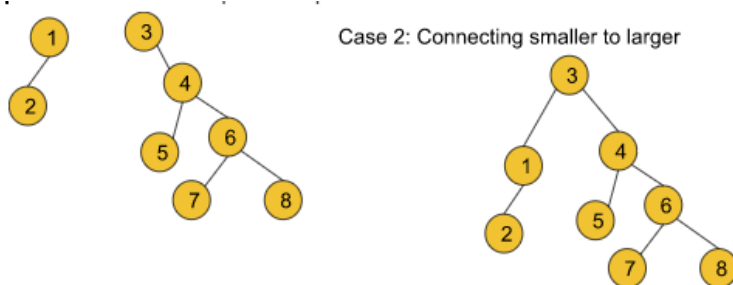
Note: The actual time complexity of union by rank and findPar() is $O(4\alpha)$ which is very small and close to 1. So, we can consider 4 as a constant. Now, this ' α ' term has a long mathematical derivation.

In the union by rank method, why do we need to connect the smaller rank to the larger rank?

- Let's understand it using the following example:



In this case, the traversal time to find the ultimate parent for nodes 3, 4, 5, 6, 7, and 8 increases and so the path compression time also increases. But if we do the following



- the traversal time to find the ultimate parent for only nodes 1 and 2 increases. So, the path compression time becomes relatively lesser than in the previous case. So, we can conclude that we should always connect a smaller rank to a larger one with the goal of
 - ***shrinking the height of the graph.***
 - ***reducing the time complexity as much as we can.***

Observation 3:

Until now, we have learned union by rank, the findPar() function, and the path compression technique. Now, if we again carefully observe, after applying path compression the rank of the graphs becomes distorted. So, rather than storing the rank, we can just store the size of the components for comparing which component is greater or smaller.

So, here comes the concept of ***Union by size.***

Union by size:

This is as same as the Union by rank method except this method uses the size to compare the components while connecting. That is why we need a '**size**' array of size N (no. of nodes) instead of a **rank** array. The size array will be storing the size for each particular node i.e. size[i] will be the size of the component starting from node i.

Typically, the size of a node refers to the number of nodes that are connected to it.

size array: This array is initialized with one.

parent array: The array is initialized with the value of nodes i.e. parent[i] = i.

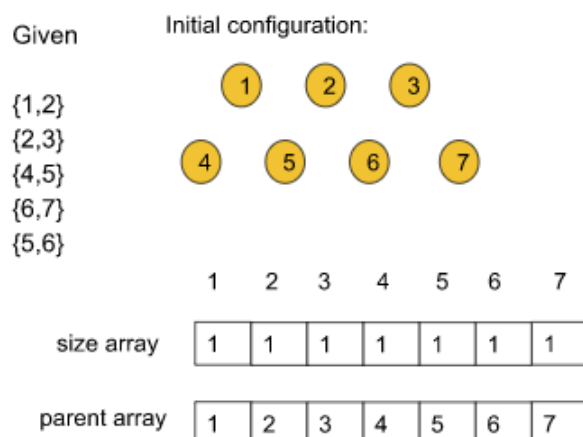
Algorithm:

1. Firstly, the Union function requires two nodes (*let's say u and v*) as arguments. Then we will find the ultimate parent (using the findPar() function discussed earlier) of u and v. Let's consider the ultimate parent of u is **pu** and the ultimate parent of v is **pv**.
2. After that, we will find the size of **pu** and **pv** i.e. size[pu] and size[pv].
3. Finally, we will connect the ultimate parent with a smaller size to the other ultimate parent with a larger size. But if the size of the two is equal, we can connect any parent to the other parent.

While connecting in both cases we will increase the size of the parent node to whom we have connected by the size of the other parent node which is actually connected.

Let's understand it further using the below example.

Given the edges of a graph are {{1,2}, {2,3}, {4,5}, {6,7}, {5,6}, {3,7}}

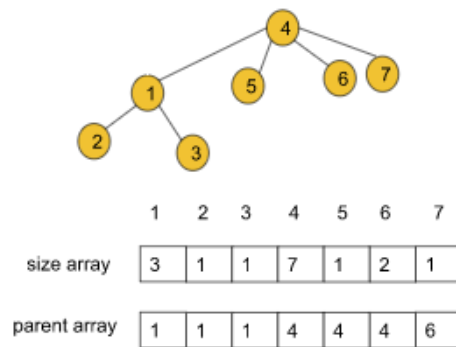


After applying the union by size function to every edge the graph and the arrays will look like the following:

Given

Union(1,2)
Union(2,3)
Union(4,5)
Union(6,7)
Union(5,6)
Union(3,7)

Final configuration:



Note: It seems much more intuitive than union by rank as the rank gets distorted after path compression.

Note: The findPar() function remains the exact same as we have discussed earlier

```
static class DisjointSet { 2 usages
    List<Integer> size = new ArrayList<>(); 5 usages
    List<Integer> parent = new ArrayList<>(); 7 usages
    DisjointSet(int n) { 1 usage
        for (int i = 0; i < n; i++) {
            size.add(1);
            parent.add(i);
        }
    }
    int findUltimateParent(int node){ 5 usages
        if (node == parent.get(node)) return node;
        int up = findUltimateParent(parent.get(node));
        parent.set(node, up);
        return parent.get(node);
    }
    void unionBySize(int u, int v){ 5 usages
        int ultimateU = findUltimateParent(u); int ultimateV = findUltimateParent(v);
        if (ultimateV == ultimateU) return;

        int sizeU = size.get(ultimateU); int sizeV = size.get(ultimateV);
        if (sizeV < sizeU){
            parent.set(ultimateV, ultimateU);
            size.set(ultimateU, sizeU + sizeV);
        }
        else{
            parent.set(ultimateU, ultimateV);
            size.set(ultimateV, sizeV + sizeU);
        }
    }
}
```


Given a weighted, undirected, and connected graph with V vertices and E edges, your task is to find the sum of the weights of the edges in the Minimum Spanning Tree (MST) of the graph. The graph is provided as a list of edges, where each edge is represented as $[u, v, w]$, indicating an edge between vertex u and vertex v with edge weight w .

Intuition

Kruskal's Algorithm is a **greedy** algorithm used to find the **(MST)**.

- Always pick the edge with the smallest weight
 - Add it only if it does NOT form a cycle
 - Use Disjoint Set (Union-Find) to efficiently detect cycles
 - Keep adding edges until all nodes are connected
-

Algorithm

1. **Initialize Disjoint Set (DSU)**
 - Each node is its own parent
 - Size of each set = 1
 2. **Sort all edges by weight (ascending)**
 3. **Traverse sorted edges**
For each edge (u, v, w) :
 - Find ultimate parents of u and v
 - If they belong to different sets:
 - Add edge to MST
 - Add w to total weight
 - Union the two sets using union by size
 4. **Result**
 - Total weight of MST
 - List of edges in MST
-

```

static void optimal(int n, int[][] edges){ 1 usage  Ⓜ ADITYA PA
    Arrays.sort(edges, (int[] a, int[] b) -> a[2] - b[2]);

    DisjointSet d = new DisjointSet(n);
    List<List<Integer>> connectedEdges = new ArrayList<>();
    int totalWeight = 0;
    for (int i=0; i<edges.length; i++){
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];

        int ultimateU = d.findUltimateParent(u);
        int ultimateV = d.findUltimateParent(v);

        if(ultimateU != ultimateV){
            d.unionByRank(edges[i][0], edges[i][1]);
            totalWeight += w;
            List<Integer> temp = new ArrayList<>();
            temp.add(u);
            temp.add(v);
            connectedEdges.add(temp);
        }
    }

    System.out.println(totalWeight);
    System.out.println(connectedEdges);
}

```

◆ Time & Space Complexity

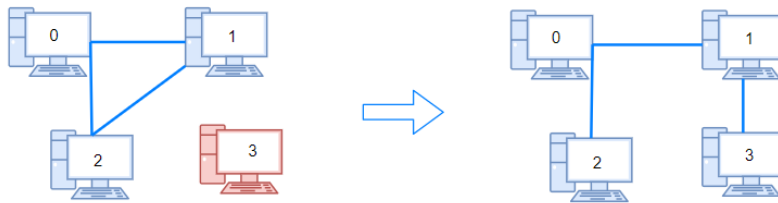
- **Sorting edges:** $O(E \log E)$
- **DSU operations:** $O(E \alpha(N))$ (almost constant)
- **Overall Time:** $O(E \log E)$
- **Space:** $O(N + E)$

There are 'n' computers numbered from 0 to n - 1 connected by ethernet cables connections forming a network where connections[i] = [a_i, b_i] represents a

connection between computers a_i and b_i . Any computer can reach any other computer directly or indirectly through the network.

You are given an initial computer network connection. You can extract certain cables between two directly connected computers, and place them between any pair of disconnected computers to make them directly connected.

Return the minimum number of times you need to do this in order to make all the computers connected. If it is not possible, return -1.



Input: $n = 4$, connections = $[[0,1],[0,2],[1,2]]$

Output: 1

Explanation: Remove cable between computer 1 and 2 and place between computers 1 and 3.

Approach:

- Use Disjoint Set (Union-Find) to track connected components.
- If no of edges < number of nodes -1 return -1.
- Initialise two variables totalComponents & extraEdges.
- Traverse all given connections:
 - If two computers are already in the same component, count that edge as an extra (redundant) cable.
 - Otherwise, union the two components & reduce totalComponent-1.
- If the number of extra cables $\geq$ (components - 1): Return components - 1
- Otherwise: Return -1 (not possible to connect all computers).

```

static int optimal(int n, int[][] edges){ no usages  @ADITYA PATEL *
    if(edges.length < n-1){
        return -1;
    }

    DisjointSet obj = new DisjointSet(n);
    int extraEdges = 0;
    int totalComponents = n;
    for (int i=0;i< edges.length;i++){
        int u = edges[i][0];
        int v = edges[i][1];
        if(obj.findUltimateParent(u) == obj.findUltimateParent(v)){
            extraEdges+=1;
        }
        else {
            obj.unionBySize(u,v);
            totalComponents -= 1;
        }
    }
    if (extraEdges >= totalComponents-1) {
        return totalComponents-1;
    }
    return -1;
}

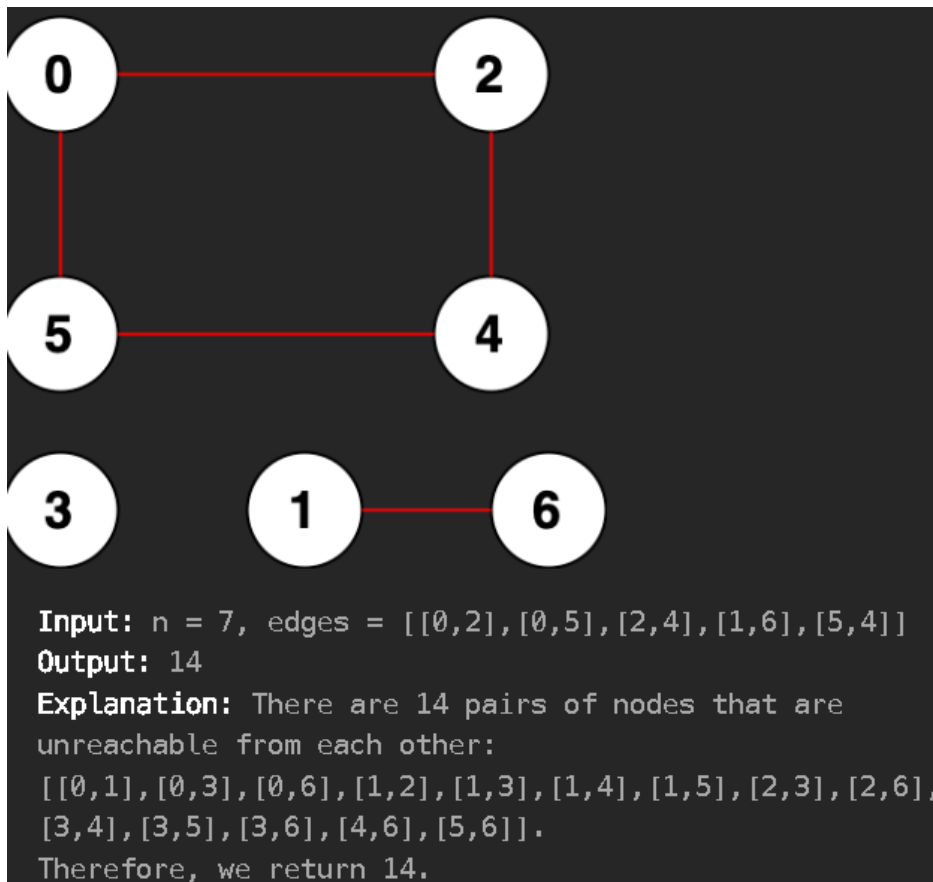
```

Time & Space Complexity

- Time: $O(E \alpha(N))$
- Space: $O(N)$

You are given an integer n . There is an undirected graph with n nodes, numbered from 0 to $n - 1$. You are given a 2D integer array `edges` where `edges[i] = [ai, bi]` denotes that there exists an undirected edge connecting nodes a_i and b_i .

Return the number of pairs of different nodes that are unreachable from each other.



Approach-1: Using bfs or dfs

- **Build the graph** using an adjacency list from the given edges.
- **Initialize** a visited array to keep track of visited nodes.
- **Traverse all nodes:**
 - If a node is unvisited, start a **DFS** from it.
- **During DFS**, visit all reachable nodes and count the number of nodes in that connected component.
- **After finding a component:**
 - Add its size to the number of nodes processed so far (total).
 - Count pairs between this component and all remaining unprocessed nodes:
 $componentSize \times (n - total)$
 - Add this value to the answer.
- **Repeat** until all components are processed.
- **Return the total count** of unreachable pairs.

Time Complexity

- $O(V + E)$

Space Complexity

- $O(V + E)$ for the adjacency list and visited array.
- $O(V)$ extra recursion stack in the worst case (DFS).

Approach-2: Using DSU

- Initialize DSU so every node starts in its own component.
- Process all edges and merge the two endpoints into the same component using union by size.
- After all unions, each connected component is represented by its ultimate parent.
- Traverse all nodes and consider only the ultimate parents (component representatives).
- For each component:
 - Get its size.
 - Add its size to the number of nodes processed so far (total).
 - Count pairs between this component and all remaining nodes:

$$\text{componentSize} \times (n - \text{total})$$

- Add this value to the answer.
- Return the total count of unreachable pairs.

Time Complexity

- $O((V + E) \times \alpha(V))$, where α is the inverse Ackermann function (almost constant).

Space Complexity

- $O(V)$ for the DSU arrays (parent and size)

On a 2D plane, we place n stones at some integer coordinate points. Each coordinate point may have at most one stone.

A stone can be removed if it shares either the same row or the same column as another stone that has not been removed.

Given an array of stones of length n where $\text{stones}[i] = [x_i, y_i]$ represents the location of the i^{th} stone, return the largest possible number of stones that can be removed.

Input: stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]

Output: 5

Explanation: One way to remove 5 stones is as follows:

1. Remove stone [2,2] because it shares the same row as [2,1].
 2. Remove stone [2,1] because it shares the same column as [0,1].
 3. Remove stone [1,2] because it shares the same row as [1,0].
 4. Remove stone [1,0] because it shares the same column as [0,0].
 5. Remove stone [0,1] because it shares the same row as [0,0].
- Stone [0,0] cannot be removed since it does not share a row/column with another stone still on the plane.

Intuition

- Think of each **stone as a node** in a graph.
- Two stones are connected if they lie in the **same row or the same column**.
- All stones that are directly or indirectly connected form a **connected component**.
- In one connected component containing k stones, we can remove **k - 1 stones**, leaving only one stone behind.
- Therefore, if there are C connected components and N stones, the maximum stones that can be removed is:

$$N - C$$

Using BFS/DFS:

- Consider each stone as a graph node.
- Connect two stones if they share the same row or the same column.
- Traverse the graph using **DFS** to find all connected components.
- Count the total number of connected components.
- Return **Total Stones – Number of Connected Components**.

Complexity Analysis

- **Time Complexity: $O(N^2)$**
 - For each DFS call, every stone may be checked against every other stone to determine if they share a row or column.
- **Space Complexity: $O(N)$**
 - $O(N)$ for the visited array.

- $O(N)$ recursion stack in the worst case (when all stones belong to one connected component).

Using DSU: Approach-1

- Initialize a **Disjoint Set (DSU)** with one node for each stone.
- Compare every pair of stones.
- If two stones share the **same row or the same column**, merge their sets using **Union**.
- After processing all pairs, count the number of **unique parents (connected components)** in the DSU.
- Compute the answer as:
 - **Total Stones – Number of Connected Components.**
- Return the computed answer.

```
public static int removeStones(int[][] stones) {
    int n = stones.length;
    DisjointSet d = new DisjointSet(n);
    for (int u = 0; u < n; u++) {
        int r = stones[u][0];
        int c = stones[u][1];
        for (int v = u + 1; v < n; v++) {
            if (stones[v][0] == r || stones[v][1] == c) {
                d.unionByRank(u, v);
            }
        }
    }
    int totalComp = 0;
    for (int i = 0; i < n; i++) {
        if (d.findUltimateParent(i) == i) {
            totalComp++;
        }
    }
    return n - totalComp;
}
```

Complexity Analysis

- Time Complexity: $O(N^2 \cdot \alpha(N))$
 - There are $O(N^2)$ pairs of stones.

- Each union/find operation takes $\alpha(N)$ (inverse Ackermann function), which is practically constant.
- **Space Complexity: $O(N)$**
 - $O(N)$ for the DSU parent and rank arrays.

Using DSU: Approach-2

- Find the **maximum row** and **maximum column** index.
- Create a DSU large enough to represent **all row nodes and all column nodes**.
- Shift the column indices so that **row and column nodes have unique IDs**.
- For every stone:
 - Treat its **row** and **column** as two DSU nodes.
 - **Union** the corresponding row node and shifted column node.
- After processing all stones, count the number of **connected components that actually contain stones** (ignore row/column nodes having size as zero).
- Compute the answer as:
 - **Total Stones – Number of Connected Components.**
- Return the result.

```

static int approach2(int[][] stones) { 1 usage  👤 ADITYA PATEL
    int n = stones.length;
    int totalRows = 0;
    int totalCols = 0;
    for (int i = 0; i < n; i++) {
        totalRows = Math.max(totalRows, stones[i][0]);
        totalCols = Math.max(totalCols, stones[i][1]);
    }

    int m = totalRows + totalCols + 2;
    DisjointSet obj = new DisjointSet(m);
    for (int i = 0; i < n; i++) {
        int row = stones[i][0];
        int col = stones[i][1] + totalRows + 1;
        obj.unionBySize(row, col);
    }

    int totalComp = 0;
    for (int i = 0; i < m; i++) {
        int ultimate = obj.findUltimateParent(i);
        if (ultimate == i && obj.size[ultimate] > 1) {
            totalComp += 1;
        }
    }

    return stones.length - totalComp;
}

```

Complexity Analysis

- **Time Complexity: $O(N \cdot \alpha(R + C))$**
 - N = number of stones
 - R = maximum row index
 - C = maximum column index
 - Each union/find operation takes $\alpha(R + C)$, which is nearly constant.
- **Space Complexity: $2 \times O(R + C)$**
 - For the DSU parent and size arrays.

Given a list of accounts where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are emails representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

```
Input: accounts =
[["Gabe","Gabe0@m.co","Gabe3@m.co","Gabe1@m.co"],
["Kevin","Kevin3@m.co","Kevin5@m.co","Kevin0@m.co"],
["Ethan","Ethan5@m.co","Ethan4@m.co","Ethan0@m.co"],
["Hanzo","Hanzo3@m.co","Hanzo1@m.co","Hanzo0@m.co"],
["Fern","Fern5@m.co","Fern1@m.co","Fern0@m.co"]]
Output: [["Ethan","Ethan0@m.co","Ethan4@m.co","Ethan5@m.co"],
["Gabe","Gabe0@m.co","Gabe1@m.co","Gabe3@m.co"],
["Hanzo","Hanzo0@m.co","Hanzo1@m.co","Hanzo3@m.co"],
["Kevin","Kevin0@m.co","Kevin3@m.co","Kevin5@m.co"],
["Fern","Fern0@m.co","Fern1@m.co","Fern5@m.co"]]
```

4

Approach:

- Each account is treated as a node.
- Accounts sharing at least one common email belong to the same person.
- Use Disjoint Set Union (DSU) to group account indices that share emails.
- After grouping, collect all emails belonging to the same root account and sort them.

```

static List<List<String>> optimal(List<List<String>> accounts) {
    int n = accounts.size();
    Disjoint obj = new Disjoint(n);
    HashMap<String,Integer> mp = new HashMap<>();
    for(int i=0;i<n;i++){
        for(int j=1;j<accounts.get(i).size();j++){
            String mail = accounts.get(i).get(j);
            if(!mp.containsKey(mail)) mp.put(mail,i);
            else obj.unionBySize(i,mp.get(mail));
        }
    }

    ArrayList<String>[] merged = new ArrayList[n];
    for(int i=0;i<n;i++) merged[i] = new ArrayList<>();
    for(Map.Entry<String,Integer> e: mp.entrySet()){
        String mail = e.getKey();
        int ind = obj.findUltimateParent(e.getValue());
        merged[ind].add(mail);
    }
    List<List<String>> ans = new ArrayList<>();
    for(int i=0;i<n;i++){
        if(merged[i].isEmpty()) continue;
        Collections.sort(merged[i]);
        List<String> temp = new ArrayList<>();
        temp.add(accounts.get(i).get(0));
        temp.addAll(merged[i]);
        ans.add(temp);
    }
    return ans;
}

```

Time Complexity = $O(E \log E)$

Let n = number of accounts, E = total number of emails.

- Scanning accounts & building map: $O(E)$
- DSU union/find operations (with path compression): $O(E \alpha(n))$
- Grouping emails by parent: $O(E \alpha(n))$
- Sorting emails in merged accounts: $O(E \log E)$

Space Complexity = $O(n + E)$

- DSU parent + size arrays: $O(n)$
- HashMap for emails: $O(E)$

- Merged email storage: $O(E)$

You are given a n, m which means the row and column of the 2D matrix and an array of size k denoting the number of operations. Matrix elements is 0 if there is water or 1 if there is land. Originally, the 2D matrix is all 0 which means there is no land in the matrix. The array has k operator(s) and each operator has two integer $A[i][0]$, $A[i][1]$ means that you can change the cell matrix $A[A[i][0]][A[i][1]]$ from sea to island. Return how many island are there in the matrix after each operation. You need to return an array of size k .

Note : An island means group of 1s such that they share a common side.

Input:

$n = 4$

$m = 5$

$k = 4$

$A = \{\{1,1\}, \{0,1\}, \{3,3\}, \{3,4\}\}$

Output: 1 1 2 2

Approach:

- We treat each cell (r, c) as a **node** with index $r*m + c$.
- **Disjoint Set Union (DSU)** is used to efficiently:
 - Track connected components (islands)
 - Merge neighbouring land cells
- For every operation:
 - If the cell is already land $\rightarrow$ island count remains same
 - Otherwise:
 - Mark it as land and increase island count by 1
 - Check its **4-directional neighbors**
 - If a neighbour is land and belongs to a **different component**, merge them using **union by size**
 - Each successful union reduces island count by 1
- After each operation, current island count is added to the answer.

```

public List<Integer> numOfIslands(int rows, int cols, int[][] mat) {
    int[] rowsIdx = {0,-1,0,1};
    int[] colsIdx = {-1,0,1,0};

    int[][] vis = new int[rows][cols];
    int totalComponent = 0;
    List<Integer> ans = new ArrayList<>();
    DisjointSet d = new DisjointSet(rows*cols);

    for(int i=0;i<mat.length;i++){
        int r = mat[i][0];
        int c = mat[i][1];
        if(vis[r][c]==0){
            vis[r][c]=1;
            totalComponent += 1;

            for(int j=0;j<4;j++){
                int nr = r+rowsIdx[j];
                int nc = c+colsIdx[j];

                if(nr>=0 && nr<rows && nc>=0 && nc<cols && vis[nr][nc]==1){
                    int u = cols*r+c;
                    int v = cols*nr + nc;

                    int ultimateU = d.findUltimateParent(u);
                    int ultimateV = d.findUltimateParent(v);

                    if(ultimateU != ultimateV){
                        d.unionByRank(u,v);
                        totalComponent -= 1;
                    }
                }
            }
            ans.add(totalComponent);
        }
    }
    return ans;
}

```

Time Complexity

Let k be the number of operations and $N = n \times m$ total cells.

For each operation, we check **4 neighbors ($O(1)$)** and perform **DSU find/union** operations which take **$O(\alpha(N))$** due to path compression and union by size (α is inverse Ackermann, nearly constant).

$$TC = O(k \cdot \alpha(n \cdot m)) \approx O(k)$$

Space Complexity

- parent[] and size[] arrays in DSU $\rightarrow O(n \cdot m)$
- vis[][] grid $\rightarrow O(n \cdot m)$

- Answer list $\rightarrow O(k)$

$$SC = O(n \cdot m)$$

You are given an $n \times n$ binary matrix grid. You are allowed to change at most one 0 to be 1.

Return the size of the largest island in grid after applying this operation.

An island is a 4-directionally connected group of 1s.

Input: grid = [[1,0],[0,1]]

Output: 3

Explanation: Change one 0 to 1 and connect two 1s, then we get an island with area = 3.

Approach:

- Treat each cell of the grid as a node and use **Disjoint Set Union (DSU)** to connect all adjacent 1 cells.
- While traversing the grid, union every 1 cell with its 4-directional neighbouring 1s, forming connected components (islands) and storing their sizes.
- Store all 0 cells separately.
- If there is no 0 in the grid, the entire grid is already one island, so return $n \times n$.
- For each 0 cell, assume it is flipped to 1 and check its 4 neighbours.
- Collect the **unique parent components** of neighbouring 1s using a Set to avoid double counting.
- The possible island size after flipping that 0 is
1 (current cell) + sum of sizes of all unique neighbouring components.
- Track the maximum possible size.

Time Complexity: $O(n^2 \cdot \alpha(n^2)) \approx O(n^2)$

- Traversing the grid and performing DSU unions.
- For each 0, checking at most 4 neighbours.
- α is the inverse Ackermann function (almost constant).

Space Complexity: $O(n^2)$

- DSU parent and size lists of size n^2 .

- List storing positions of 0 cells.
- Temporary Set of at most 4 elements per 0 cell.

You are given an $n \times n$ integer matrix grid where each value $\text{grid}[i][j]$ represents the elevation at that point (i, j) .

It starts raining, and water gradually rises over time. At time t , the water level is t , meaning any cell with elevation less than equal to t is submerged or reachable.

You can swim from a square to another 4-directionally adjacent square if and only if the elevation of both squares individually are at most t . You can swim infinite distances in zero time. Of course, you must stay within the boundaries of the grid during your swim.

Return the minimum time until you can reach the bottom right square $(n - 1, n - 1)$ if you start at the top left square $(0, 0)$.

0	2
1	3

Input: $\text{grid} = [[0,2], [1,3]]$

Output: 3

Explanation:

At time 0, you are in grid location $(0, 0)$.

You cannot go anywhere else because 4-directionally adjacent neighbors have a higher elevation than $t = 0$.

You cannot reach point $(1, 1)$ until time 3.

When the depth of water is 3, we can swim anywhere inside the grid.

Approach:

Think of this problem as finding a path where the highest elevation you ever step on is as small as possible.

- Water level must be at least as high as the maximum cell elevation on your path.

- So, the goal is not shortest distance, but minimum “maximum height” along the path.

This code uses a min-heap (priority queue) to always move to the currently lowest possible elevation cell, similar to Dijkstra’s algorithm:

- Start from (0,0)
- Always explore the neighbour with the smallest height first
- Keep track of the maximum height encountered so far (ans)
- The moment you reach the bottom-right cell, that maximum height is the minimum time needed

```
static int optimal(int[][] grid) { 1 usage
    int n = grid.length;

    int ans = Integer.MIN_VALUE;
    int[][] vis = new int[n][n];
    PriorityQueue

```

Time Complexity

- The grid has $n \times n = n^2$ cells
- Each cell is inserted into the priority queue once
- Each insert / remove operation from the priority queue costs $O(\log(n^2)) = O(\log n)$

$$O(n^2 \log n)$$

Space Complexity

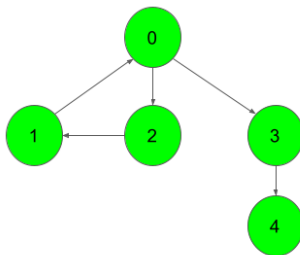
- **Visited array vis** → stores $n \times n$ cells → $O(n^2)$
- **Priority Queue** → in worst case can hold all grid cells → $O(n^2)$

$$O(n^2)$$

Other Algorithms

Given a Directed Graph with V vertices (Numbered from 0 to $V-1$) and E edges. The graph is represented as a 2D vector `edges[][]`, where each entry `edges[i] = [u, v]` denotes a direct edge from vertex u to v . Find the number of strongly connected components in the graph.

Input: `adj[][] = [[2, 3], [0], [1], [4], []]`



Output: 3

Intuition (Kosaraju's Algorithm): Used to find SCC in a directed graph only

- An SCC (Strongly Connected Component) is a group of vertices where every vertex can reach every other vertex.
- The key idea is to process the graph in an order such that when we start a DFS, we always enter an SCC before entering any other SCC connected to it.
- Step 1: Perform a DFS on the original graph and store vertices according to their finishing time (the vertex that finishes last is placed on the top of the stack that is topological sort). This ensures that SCCs with no outgoing edges are processed first in the next phase.
- Step 2: Reverse all the edges of the graph. Reversing the graph changes the direction between SCCs but does not break the internal connectivity of an SCC.

- Step 3: Pop vertices from the stack one by one. Whenever an unvisited vertex is found, perform DFS on the reversed graph. This DFS visits exactly one SCC, because the reversed edges prevent the traversal from moving into another SCC.
- Repeat until the stack becomes empty. The number of DFS calls in the reversed graph equals the number of Strongly Connected Components (SCCs).

Key Insight

- Finishing times tell us the correct order in which SCCs should be processed.
- Reversing the graph isolates each SCC during DFS.
- Each DFS in the reversed graph discovers exactly one SCC.

Complexity Analysis

Time: $O(V + E)$

Reason:

1. **Build the adjacency list:** $O(E)$
2. **First DFS (to compute finishing times):** $O(V + E)$
3. **Reverse the graph:** $O(E)$
4. **Second DFS on the reversed graph:** $O(V + E)$

Overall:

$$O(E) + O(V + E) + O(E) + O(V + E) = O(V + E)$$

Since constants are ignored,

$$\boxed{O(V + E)}$$

Space: $O(V + E)$

- Adjacency list: $O(V + E)$
- Reversed adjacency list: $O(V + E)$
- Visited array: $O(V)$
- Stack (stores finishing order): $O(V)$
- DFS recursion stack (worst case): $O(V)$

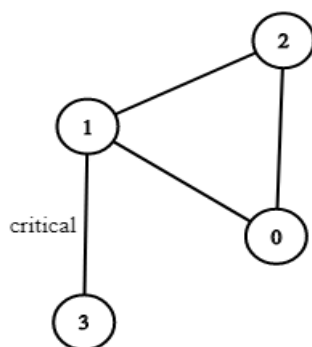
Overall:

$$O(V + E)$$

There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server connections forming a network where $\text{connections}[i] = [a_i, b_i]$ represents a connection between servers a_i and b_i . Any server can reach other servers directly or indirectly through the network.

A critical connection is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.



Input: $n = 4$, $\text{connections} = [[0,1],[1,2],[2,0],[1,3]]$

Output: $[[1,3]]$

Explanation: $[[3,1]]$ is also accepted.

Brute-force: Using edge removal

A critical connection (bridge) is an edge whose removal disconnects the graph.

The most straightforward idea is to check every edge individually.

Steps

1. Iterate through every edge in the graph.
2. Temporarily remove the current edge.
3. Perform a DFS/BFS from any node (e.g., node 0) on the remaining graph.
4. Check whether all n nodes are visited.

- If **all nodes are reachable**, removing this edge does **not** disconnect the graph, so it is **not** a critical connection.
 - If **some nodes are unreachable**, removing this edge disconnects the graph, so it **is** a critical connection.
5. Restore the removed edge and repeat the process for the next edge.

Brute-force: Using cycle detection

A critical connection (bridge) is an edge that **does not belong to any cycle**.

So, for every edge, we can check whether there exists **another path** between its two endpoints after ignoring that edge.

Steps

1. Iterate through every edge (u, v) in the graph.
2. Temporarily ignore/remove the edge (u, v) .
3. Start a DFS/BFS from node u and try to reach node v .
4. If v is still reachable, it means there is another path from u to v , so the edge is part of a cycle and **is not a critical connection**.
5. If v is **not** reachable, then the removed edge was the only path between u and v . Hence, it is **not part of any cycle** and **is a critical connection**.
6. Repeat this process for every edge.

Complexity

- Let V = number of vertices and E = number of edges.
- For each of the E edges, we perform one DFS/BFS taking $O(V + E)$ time.

Time Complexity: $O(E \times (V + E))$

Space Complexity: $O(V + E)$ (adjacency list + visited array)

Optimal (Tarjan's Algorithm for Bridges)

Intuition

A critical connection (bridge) is an edge whose removal disconnects the graph. During DFS, for each node we store:

- Insertion Time (tin): The time when the node is first visited.
- Lowest Time (low): The earliest visited node reachable from the current node or its subtree using tree edges and at most one back edge.

If a child's subtree cannot reach the current node or any of its ancestors ($\text{low}[\text{child}] > \text{tin}[\text{parent}]$), then the edge between them is the only path connecting the two parts of the graph, making it a critical connection.

Steps

1. Build the adjacency list from the given connections.
 2. Initialize the visited, tin, and low arrays.
 3. Start a DFS from any unvisited node.
 4. For each unvisited neighbour, recursively perform DFS and update the current node's low value after returning.
 5. If the child's low value is greater than the current node's tin, mark the connecting edge as a critical connection.
 6. For an already visited neighbour (that is not the parent), update the current node's low value using the neighbour's insertion time.
 7. Return all the identified critical connections.
-

```
static void dfs(int curr,int parent, int timer, List<List<Integer>> ans){ 2
    visited[curr] = 1;
    timeOfInsertion[curr] = timer;
    lowestTime[curr] = timer;

    for(int nei: adj.get(curr)){
        if(nei==parent) continue;
        else if(visited[nei]==0){
            dfs(nei,curr, timer: timer+1,ans);
            lowestTime[curr] = Math.min(lowestTime[nei],lowestTime[curr]);
            if(lowestTime[nei] > timeOfInsertion[curr]){
                ans.add(new ArrayList<>(Arrays.asList(curr,nei)));
            }
        }
        else lowestTime[curr] = Math.min(lowestTime[nei],lowestTime[curr]);
    }
}
```

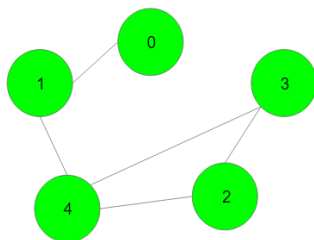
Complexity

- **Time Complexity: $O(V + E)$**
 - Each vertex is visited once and each edge is processed at most twice.
 - **Space Complexity: $O(V + E)$**
 - Adjacency list: $O(V + E)$
 - visited, tin, low arrays: $O(V)$
 - DFS recursion stack (worst case): $O(V)$
-

Given an undirected connected graph with V vertices and adjacency list `adj`. You are required to find all the vertices removing which (and edges through it) disconnects the graph into 2 or more components and return it in sorted manner.

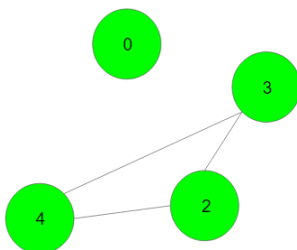
Note: Indexing is zero-based i.e nodes numbering from (0 to $V-1$). There might be loops present in the graph.

Input:

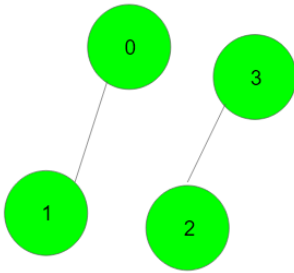


Output: {1,4}

Explanation: Removing the vertex 1 will disconnect the graph as-



Removing the vertex 4 will disconnect the graph as-



Brute-force Solution:

We will remove each node one by one from the graph along with their edges & will count number of components & whenever will get number of components > 1 then, will say that the selected node is an articulation point.

$$T.C = O(V*(V+E))$$

Optimal Solution:

Intuition

Property: A node will be not be an articulation point if all the neighbours of that node will belong to the same cycle.

An articulation point is a vertex whose removal disconnects the graph into two or more components. During DFS, for each node we store:

- Insertion Time (tin): The time when the node is first visited.
- Lowest Time (low): The earliest visited node reachable from the current node or its subtree using tree edges and at most one back edge.

A node is an articulation point if any of its children's subtrees cannot reach the node's parent or any ancestor of the node ($low[child] \geq tin[node]$). This means the current node is the only connection between that subtree and the rest of the graph.

Special Case (Root Node):

The root of the DFS tree is an articulation point only if it has more than one child, because removing it separates those child subtrees.

Steps

1. Build the adjacency list from the given graph.
2. Initialise the visited, tin, and low arrays.

3. Start a DFS from every unvisited node (to handle disconnected graphs).
4. For every unvisited **neighbour**, recursively perform DFS and update the current node's low value after returning.
5. If the current node is **not the root** and a **neighbour's** low value is greater than or equal to the current node's tin, mark the current node as an articulation point.
6. For every already visited **neighbour** (excluding the parent), update the current node's low value using the **neighbour's** insertion time.
7. After processing all **neighbours**, if the current node is the **root** and has more than one DFS child, mark it as an articulation point.
8. Return all the articulation points in sorted order. If none exist, return -1.

```
static void dfs(int curr,int parent, int timer,boolean[] articulationPoint){
    visited[curr] = 1;
    timeOfInsertion[curr] = timer;
    lowestTime[curr] = timer;

    int child=0;
    for(int nei: adj.get(curr)){
        if(nei==parent) continue;
        else if(visited[nei]==0){
            dfs(nei,curr, timer+1,articulationPoint);
            lowestTime[curr] = Math.min(lowestTime[nei],lowestTime[curr]);
            if(lowestTime[nei] >= timeOfInsertion[curr] && parent != -1 ){
                articulationPoint[curr] = true;
            }
            child++;
        }
        else {
            lowestTime[curr] = Math.min(timeOfInsertion[nei],lowestTime[curr]);
        }
    }
    if(parent==-1 && child>1){
        articulationPoint[curr] = true;
    }
}
```

Complexity

- **Time Complexity: $O(V + E)$**
 - Each vertex is visited once, and each edge is processed at most twice.
- **Space Complexity: $O(V + E)$**

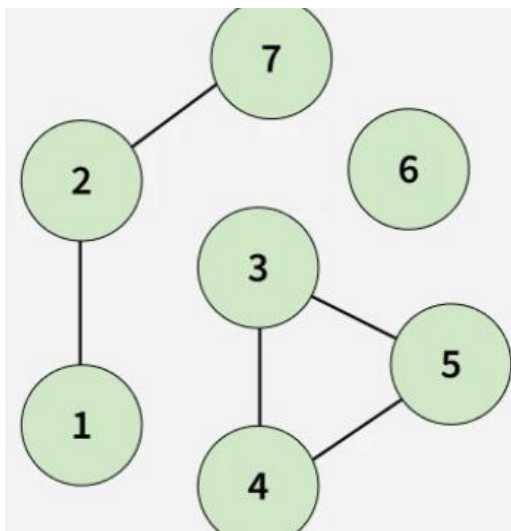
- Adjacency list: $O(V + E)$
- visited, tin, low, and articulation point arrays: $O(V)$
- DFS recursion stack (worst case): $O(V)$

Given an undirected graph with v vertices (numbered from 1 to v) and e edges. Find the number of good components in the graph.

A component of the graph is good if and only if the component is fully connected.

Note: A fully connected component is a subgraph of a given graph such that there's an edge between every pair of vertices in the component, the given graph can be a disconnected graph.

Input:



Output: 2

Explanation:

We can see that there are 3 components in the graph. For 1-2-7 there is no edge between 1 to 7, so it is not a fully connected component. Rest 2 are individually fully connected component.

Intuition

A good component is a connected component in which every vertex is directly connected to every other vertex (a complete graph).

In a complete graph with k vertices, every vertex must have exactly $k - 1$ neighbours.

Therefore, after finding each connected component, we simply check whether every vertex in that component has degree size - 1. If yes, the component is good; otherwise, it is not.

Steps

1. Build the adjacency list and calculate the degree of every vertex.
2. Initialise the visited array.
3. Traverse the graph and perform a DFS from every unvisited vertex to find each connected component.
4. Store all the vertices belonging to the current component.
5. For each component, determine its size.
6. Check whether every vertex in the component has degree equal to size - 1.
7. If all vertices satisfy the condition, count the component as a good component.
8. Return the total number of good components.

Complexity

- **Time Complexity: $O(V + E)$**
 - Building the adjacency list and degree array: $O(E)$
 - DFS traversal: $O(V + E)$
 - Checking all components: $O(V)$
 - Overall: $O(V + E)$
- **Space Complexity: $O(V + E)$**
 - Adjacency list: $O(V + E)$
 - Visited array: $O(V)$
 - Component storage: $O(V)$
 - DFS recursion stack (worst case): $O(V)$

Eulerian Path (Undirected Graph)

- An Eulerian Path (Euler Trail) is a path that visits every edge exactly once. A vertex may be visited multiple times.

- Not every graph has an Eulerian Path.
- If an Eulerian Path exists but not an Eulerian Circuit:
 - The starting and ending vertices must have odd degree.
 - All other vertices must have even degree.
- All vertices with degree > 0 must belong to the same connected component (vertices with degree 0 are ignored).
- A graph that has an Eulerian Path but not an Eulerian Circuit is called a Semi-Eulerian Graph.

Directed Graph

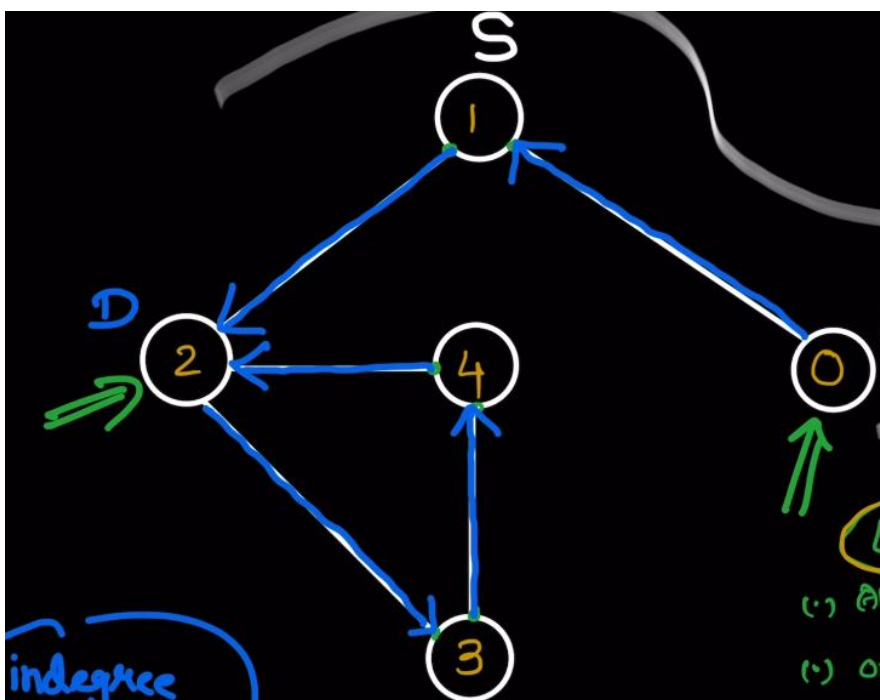
- Every vertex except two must satisfy:
 - In-degree = Out-degree
- Start vertex: Out-degree - In-degree = 1
- End vertex: In-degree - Out-degree = 1
- All vertices having non-zero degree must belong to the same connected component (when edge directions are ignored, i.e., the underlying undirected graph is connected).

Eulerian Circuit (Undirected Graph)

- An Eulerian Circuit (Eulerian Cycle) is an Eulerian Path that starts and ends at the same vertex.
- If a graph has an Eulerian Circuit, the traversal can start from any vertex with a non-zero degree.
- A graph has an Eulerian Circuit if and only if:
 - All vertices have even degree.
 - All vertices with degree > 0 belong to the same connected component.
- A graph having both an Eulerian Path and an Eulerian Circuit is called an Eulerian Graph.
- Every graph having an Eulerian Circuit also has an Eulerian Path, but the converse is not true.

Directed Graph

- Every vertex must satisfy:
 - In-degree = Out-degree
- All vertices having non-zero degree must belong to a single strongly connected component (ignoring isolated vertices).
- If these conditions hold, an Eulerian Circuit exists and the traversal can start from any vertex with a non-zero degree.



An Eulerian Path is a path in graph that visits every edge exactly once. An Eulerian Circuit is an Eulerian Path which starts and ends on the same vertex. Given an undirected graph with V nodes, and E edges, with adjacency list adj , where $adj[i]$ stores all the nodes that have an edge with i , return 2 if the graph contains an eulerian circuit, else if the graph contains an eulerian path, return 1, otherwise, return 0.

Intuition

- An Eulerian traversal is possible only if all edges belong to a single connected component. Therefore, first verify that all vertices with non-zero degree are connected using DFS (ignore isolated vertices).

- Once connectivity is confirmed, the existence of an Eulerian Path or Circuit depends only on the degree of the vertices.
- If every vertex has even degree, we can enter and leave every vertex the same number of times, so the traversal can start and end at the same vertex. Hence, an Eulerian Circuit exists.
- If exactly two vertices have odd degree, one must be the starting vertex and the other the ending vertex, while all other vertices have even degree. Hence, only an Eulerian Path exists.
- If more than two vertices have odd degree, it is impossible to traverse every edge exactly once.

Key Idea

- Connectivity ensures that every edge is reachable.
- Vertex degrees determine whether the traversal forms:
 - an Eulerian Circuit (0 odd-degree vertices),
 - an Eulerian Path (2 odd-degree vertices), or
 - neither (more than 2 odd-degree vertices).

Complexity Analysis

Time Complexity: $O(V + E)$

- Finding a vertex with non-zero degree: $O(V)$
- DFS traversal: $O(V + E)$
- Checking connectivity of non-zero degree vertices: $O(V)$
- Counting vertices with odd degree: $O(V)$

Overall:

$$O(V) + O(V + E) + O(V) + O(V) = \boxed{O(V + E)}$$

Space Complexity: $O(V)$

- Visited array: $O(V)$
- DFS recursion stack (worst case): $O(V)$

Overall:

$$O(V)$$

How to print Eulerian path?

Intuition

Treat each pair as a **directed edge**. We need to arrange the pairs so that the end of one pair is the start of the next, which is exactly finding an **Eulerian Path** (a path that uses every edge exactly once).

Use **Hierholzer's Algorithm** to construct the Eulerian path by traversing edges until no more outgoing edges remain, then backtrack. The path is obtained in reverse order, so reverse it to form the final arrangement.

Steps

1. Build the directed graph using an adjacency list and calculate the in-degree and out-degree of every node.
2. Find the starting node:
 - If a node has $\text{outDegree} - \text{inDegree} == 1$, start from it.
 - Otherwise, start from any node with an outgoing edge (here `pairs[0][0]`).
3. Apply **Hierholzer's Algorithm** using a stack:
 - If the current node has outgoing edges, traverse one and push the next node onto the stack.
 - Otherwise, pop the node and add it to the Eulerian path.
4. The obtained Eulerian path is in reverse order, so traverse it backwards.

Complexity:

Let **E** = number of edges (pairs) and **V** = number of vertices (distinct nodes).

Time Complexity

- Building the adjacency list and calculating in/out degrees: **$O(E)$**
- Finding the start node: **$O(V)$**

- Hierholzer's Algorithm:
 - Every edge is traversed exactly once and removed exactly once: $O(E)$
- Constructing the answer from the Eulerian path: $O(E)$

Overall Time Complexity:

$$O(V + E)$$

Since in a graph with E edges, $V \leq 2E$, it is often written as:

$$O(E)$$

Space Complexity

- Adjacency list: $O(E)$
- In-degree map: $O(V)$
- Out-degree map: $O(V)$
- Stack: $O(E)$ (in the worst case)
- Eulerian path list: $O(E)$

Auxiliary Space:

$$O(V + E)$$

Find diameter of an undirected graph. Diameter of a graph (sometimes called the width) is the number of edges on the longest path between two vertices in the graph.

If you choose a random node (u) and find the farthest node (v) from (u), then this node (v) will always be one end of the Diameter of the graph.

Consider the following graph:

A
|
B



The diameter is: $E \rightarrow C \rightarrow B \rightarrow D \rightarrow F \rightarrow H$

Length = **6 vertices (5 edges)**.

The two endpoints of the diameter are **E and H**

Suppose we randomly choose $u = A$

Distances from A:

Node	Distance
B	1
C	2
D	2
E	3
F	3
G	4
H	4

The farthest node is: H

Notice, H is indeed **one endpoint of the diameter**.

Now choose another random node.

Suppose, $u = C$

Distances:

$C \rightarrow E = 1$

$C \rightarrow H = 5$

$C \rightarrow G = 5$

The farthest node is **H**.

Again, **H** is a diameter endpoint.

Why must this happen?

Suppose the farthest node from u were **not** a diameter endpoint. Then there would exist another path extending beyond it, meaning we could travel even farther from u . That contradicts the assumption that it was the farthest node. In a graph, because there is **only one simple path between any two nodes**, the farthest node always lies at an extreme end of the tree. Hence it is an endpoint of some diameter.

The farthest node from one end of the diameter is the other end of diameter.

Using the same tree, Take **E** which is one endpoint.

Distances:

Node	Distance
C	1
B	2
D	3
F	4
G	5
H	5

The farthest node is H

which is exactly the other endpoint.

Similarly, starting from H the farthest node is E

Why is Statement 2 true?

Suppose from endpoint **E** there existed another node **X** farther than **H**. Then $E \rightarrow X$ would be longer than $E \rightarrow H$. But $E \rightarrow H$ is already the **diameter**, i.e., the longest path in the tree. This is impossible. Hence the farthest node from one endpoint **must** be the other endpoint.

Steps

1. Build the adjacency list from the given edges.
 2. Perform the **first DFS** from any node (e.g., 0) to find the farthest node. This node is one endpoint of the tree's diameter.
 3. Reset the visited array and distance.
 4. Perform the **second DFS** starting from the endpoint found in Step 2 to find the farthest node from it.
 5. The maximum distance obtained during the second DFS is the **diameter of the tree**.
-

Why does it work?

- The first DFS reaches one endpoint of the longest path (diameter).
- The second DFS starts from that endpoint and reaches the opposite endpoint.
- The distance between these two endpoints is the diameter.

Complexity

- **Time Complexity: $O(V + E)$**
 - Building the adjacency list: **$O(E)$**
 - First DFS: **$O(V + E)$**
 - Second DFS: **$O(V + E)$**
 - Overall: **$O(V + E)$**
- **Space Complexity: $O(V + E)$**
 - Adjacency list: **$O(V + E)$**
 - Visited array: **$O(V)$**
 - DFS recursion stack (worst case): **$O(V)$**